ALIŞTIRMA DEFTERİ · KOCO CİLDİ

Keyifli Alıştırmalar

Koco cildi

Programlamaya ve Algoritmalara Keyifli ve İşlevsel Bir Giriş'in alıştırma defteri. Hazırlık ve altı bölüm; her konuda dört basamak.

H Hazırlık · I İlk Adımlar · II Veri · III Soyutlama · IV Özyineleme · V Çizgeler · VI Dinamik Programlama

Çözüm yok, ipucu var.

Yanıt sizde — ya da yan masadaki arkadaşınızda.



Bülent Başaran

2026 · birinci sürüm

CC BY-SA 4.0 · kod MIT

github.com/bulent2k2/cpp_ogreniyoruz

HAZIRLIK

iKojo'da, Hiçbir Şey Yüklemeden

Koco cildinde hazırlık kısa: kaplumbağa zaten yumuşak bir giriş. Tek konu, beş alıştırma; hepsi tarayıcıda, ikojo.fly.dev'de. Amaç kitapçığın I.1 komut tablosunu elinize alıştırmak: ileri, dön, zıpla, renk. Bu cildin bütün alıştırmaları iKojo'da (ya da Koco'da) çalışır; konsola yazan alıştırmaların beklenen çıktısı var, çizenlerin ise tuvalde ne görmeniz gerektiğini anlatan bir satır.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

KURAL

YAPAY ZEKÂ İLE ÇALIŞMA KURALI

Bugün hepinizin elinde bir model var; bu defter onu yasaklamıyor, yerini söylüyor. **Isınma ve Alıştırma** basamaklarında sormayın: o sorular kas yapmak için, kası başkası çalıştıramaz. **Zorlayıcı**'da yalnızca ipucu isteyin, kod istemeyin (“hangi veri yapısı?”, “neyi gözden kaçırıyorum?”). **Keşif**'te serbest; ama yapıştırdığınız her satırı bir arkadaşınıza açıklayabilmelisiniz, açıklayamadığınız satır sizin değildir. “Yanıt yerine” kutusu da aynı ilkenin uzantısı: önce kitapçıktaki altbölüm, sonra yapay zekâ.

SÜRE

NE KADAR SÜRER?

Basamak zaten zorluğu söylüyor; süre de ona bağlı. `ISINMA` on dakika: kitapçığın programını çalıştırıp bir satırını değiştirmek. `ALIŞTIRMA` yirmi–otuz dakika: aynı kalıpla yeni, küçük bir iş. `ZORLAYICI` kırk beş–doksan dakika: kalıbı bükmek, iki kalıbı birleştirmek ya da bir hata bulmak. `KEŞİF` açık uçlu: bir akşam da sürebilir, bir hafta da. Bu ölçünün dışına taşanların başlığında bir not var: `~2 saat` gibi. Süreyi aşmak sorun değil; hiç uğraşmadan atlamak sorun.

İKİLİ

İKİLİ ÇALIŞMA VE KOD OKUMA LİSTESİ

Sürücü–seyrüseferci düzeni Hazırlık'ta kalmasın: her bölümde iki Zorlayıcı soru `İKİLİ` imini taşıyor. Onları ikili yazın; bitince kodu değişin ve birbirinizinkini şu beş soruyla okuyun. Beşine de "evet" diyemediğiniz kod bitmemiştir.

1. Adlar anlaşılır mı? `x` , `tmp` , `a2` yerine ne demek istediğini söyleyen adlar var mı?
2. Tekrar var mı? Aynı üç satır iki yerde geçiyorsa işleve çıkmalı.
3. Sınır durumu denendi mi? Boş girdi, tek öge, en büyük değer, eksi sayı.
4. Test var mı? En azından bir `assert` ya da altın dosya; "çalıştırdım, oldu" test değildir.
5. Çıktı biçimi doğru mu? Boşluk, satır sonu, sıra; `make test` sizin yerinize bakar.

H.1

Kaplumbağayı tanımak

Kitapçığın ilk sayfası: `ileri(100)` , `sağ()` ve `yinele(4)` . Beş alıştırma, hepsi çizim; kodu iKojo'ya yapıştırıp Çalıştır'a basın. Her birinden sonra bir sayıyı değiştirip yeniden çalıştırmak, bu cildin ana alışkanlığı.

2 2 1

YANIT YERİNE

kitapçık: Kılavuz (kurulum, iKojo) kitapçık I.1: Kaplumbağayla tanışın Koco'ya davet temel-bilgiler 2: çekirdek fikirler iKojo komut kılavuzu

iKojo bir şeye itiraz ettiğinde iletiyi çıktı penceresinde İngilizce gösterir ve hatalı satırı kırmızıyla çizer. İlk hafta en sık göreceğiniz on ileti, Türkçesi ve çoğu zamanki nedeni:

İLETİ	TÜRKÇESİ	ÇOĞU ZAM
`not found: value ileri`	'ileri' diye bir değer yok	Komutun Türkçe ha `sağ`).
`type mismatch; found: Int required: String`	tür uyuşmazlığı; bulunan Sayı, gereken Yazı	İşlev başk `satıryaz` girdisinin
`not enough arguments for method ileri`	ileri için girdi eksik	`ileri` y parantezi
`too many arguments for method sağ`	sağ için fazla girdi	Komut o l kitapçığın
`';' expected but ')' found`	';' bekleniyordu, ')' bulundu	Parantez düzeni bo fazla ya d
`unclosed string literal`	tırnak kapanmamış	Yazı sabit
`reassignment to val`	dez'e yeniden değer atanıyor	`dez` ile t değişmes (kitapçık
`value boy is not a member of List[Int]`	Dizin'in 'boy' diye bir üyesi yok	Yöntem a `boyu`; bi sayılır.
`recursive method f needs result type`	özyinelemeli f'nin çıktı türü yazılmalı	Kendini ç türünü si `tanım f`
`java.lang.StackOverflowError`	çağrı yığını taştı (çalışırken)	Özyineler durumu y (kitapçık

ISINMA KH-1-I1

İlk çizgi, sonra L

Kitapçığın ilk iki satırı: yukarı bir çizgi, sonra doğuya dön. Üçüncü bir satır ekleyin: kaplumbağa 50 adım daha yürüsün. Tuvalde ters bir L belirmeli. Şimdi sorun: ilk satırı `ileri(100)` yerine `geri(100)` yapınca L nereye döner? Çalıştırmadan tahmin edin, sonra çalıştırın.

ana.kojo BAŞLANGIÇ

```
// KH-1-I1 · İlk çizgi, sonra L
ileri(100)   // 100 adım ileri
sağ()        // 90 derece sağa dön
// üçüncü satır sizin
```

> **Tuvalde:** 100 adımlık dikey bir çizgi, üst ucundan sağa uzanan 50 adımlık yatay bir çizgi; kaplumbağa doğuya bakıyor.

Dizin: `alistirma-koco/kod/KH-1-I1/` — beklenen çıktı yok, çizim

ISINMA KH-1-I2

Kitapçığın sürprizi

Kare kodunda iki sayı var: 4 ve 100; bir de `sağ()`'ın gizli 90'ı. Kitapçığın "Deneyin"ini yapın: `yinele(3)` ile `sağ(120)`, `yinele(6)` ile `sağ(60)`, sonra `yinele(5)` ile `sağ(144)`. Sonuncusu neden kapalı bir şekil veriyor, üstelik beş köşeli? İpucu: dönüşlerin toplamı kaç derece? 360 değil.

ana.kojo BAŞLANGIÇ

```
// KH-1-I2 · Kitapçığın sürprizi: 4 ve 90'ı değiştirin
yinele(4) {
  ileri(100)
  sağ()
}
```

> **Tuvalde:** Beş köşeli bir yıldız; kaplumbağa başladığı yerde, başladığı yöne bakıyor (5 × 144 = 720: iki tam tur).

Dizin: `alistirma-koco/kod/KH-1-I2/` — beklenen çıktı yok, çizim

ALIŞTIRMA KH-1-A1

Renkli kare

Kitapçığın komut tablosundan dört komut: `kalemRenginiKur`, `boyamaRenginiKur`, `kalemKalınlığınıKur` ve `hızıKur`. Kareyi mavi kalemle, sarı dolguyla, kalın çizgiyle ve hızlı çizdirin. Sonra `kalemRenginiKur(rastgeleRenk)` satırını *döngünün içine* alın: dört kenar dört renk olur mu?

> **Tuvalde:** İçi sarı, kenarları kalın bir kare; döngü içindeki rastgeleRenk ile her kenar başka renk.

Dizin: `alistirma-koco/kod/KH-1-A1/` — beklenen çıktı yok, çizim

ALIŞTIRMA KH-1-A2

Zıpla ve atla

İki kare yan yana, aralarında çizgi olmadan. `zıpla(150)` kalemi kaldırıp yürür; `atla(x, y)` ise çizmeden verilen noktaya gider. Birinci kareden sonra zıplayın, ikinciyi çizin; sonra `atla(-150, 0)` ile sola gidip üçüncüyü çizin. Dikkat: zıpladıktan sonra kaplumbağa hangi yöne bakıyor? Kareler yan yana mı çıktı, yoksa biri yukarıda mı?

> **Tuvalde:** Üç eş kare yan yana (x = -150, 0, 150), aralarında çizgi yok. sağ()/sol() olmadan ikinci kare yukarıya çıkar: kaplumbağa kuzeye bakıyordu.

Dizin: `alistirma-koco/kod/KH-1-A2/` — beklenen çıktı yok, çizim

ZORLAYICI KH-1-Z1

Kesik çizgili kare

Kesik çizgi: on kere “10 adım çiz, 10 adım zıpla”. Bunu bir karenin dört kenarına uygulayın: `yinele`'nin içinde `yinele`. İç döngü bitince kaplumbağa kenarın sonunda mı? Kaç tane kısa çizgi çizildi (sayın: 4 × ?)? Sonra zıplama boyunu 5 yapıp kareyi yeniden çizin: kenar uzunluğu değişti mi, çizgi sayısı mı?

DÜŞÜNCE

Bir kenar on kısa çizgi; dört kenar, dört kere on.

YAKLAŞIM

Dıştaki `yinele(4)` kenarı, içteki `yinele(10)` çizgileri sayar; iç döngü bitince `sağ()`. Çizgi sayısı 4 × 10.

İLK SATIR

```
yinele(4) { yinele(10) { ileri(10); zıpla(10) }; sağ() }
```

Tuvalde: Kenarı 200 adım olan, her kenarı on kısa çizgiden oluşan kesik çizgili bir kare: 40 kısa çizgi. Zıplama 5 olunca kenar 150'ye iner, çizgi sayısı yine 40.

Dizin: `alistirma-koco/kod/KH-1-Z1/` — beklenen çıktı yok, çizim

BÖLÜM I

Kaplumbağayla İlk Adımlar

Kitapçığın ilk bölümü dört yapıtaşını kaplumbağayla öğretiyor: tanım, dez/den, çıktısı olan işlev, yineleme; sonunda bir oyun. Burada dört konu, on dokuz alıştırma. Çizenler tuvalde, hesaplayanlar çıktı penceresinde: ikincilerin beklenen çıktısı var, kendi çıktınızla satır satır karşılaştırın. C++ cildinin I. bölümüyle aynı sorular bazen aynı sayılarla geliyor (Project Euler 1, 2, 3, 6, 9); iki dilde yan yana koyun.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

I.1

Kendi komutunuz: tanım

`tanım kare(kenar: Sayı) = { ... }`: dört satır, ve kare artık bir komut. Kitapçığın çiçeği (on iki kare, 30'ar derece) ile sayaçlı yineleme `yineleDizinli` buradan. Bu konunun alıştırmaları çiziyor; biri de sayıyor.

1 2 1 1

> YANIT YERİNE
>
> kitapçık I.2: Kendi komutunuz temel-bilgiler 2: çekirdek fikirler (tanım) Skala kılavuzu: tanım (def) iKojo Skala kılavuzu

ISINMA KI-1-I1

Bir de üçgen

`kare` hazır. Yanına `üçgen(kenar: Sayı)` yazın: üç kere ileri, 120 derece dön. Sonra ikisiyle bir ev çizin: önce kare, üstüne üçgen çatı. Çatının yerine oturması için kareden sonra kaplumbağayı nereye getirmeli, hangi yöne çevirmeli? Kâğıda çizip düşünün; sonra kodlayın.

ana.kojo — BAŞLANGIÇ

```
// KI-1-I1 · Bir de üçgen: üçgen'i yazın, ev çizin
tanım kare(kenar: Sayı) = {
  yinele(4) {
    ileri(kenar)
    sağ()
  }
}
kare(100)
```

> **Tuvalde:** Kenarı 100 olan bir kare, üstünde kenarı 100 olan eşkenar üçgen çatı: bir ev.

Dizin: `alistirma-koco/kod/KI-1-I1/` — beklenen çıktı yok, çizim

ALIŞTIRMA KI-1-A1

Yel değirmeni, sonra çiçek

Dört kare, her biri 90 derece dönük: bir yel değirmeni. Sonra kitapçığın çiçeği gibi: on iki üçgen, 30'ar derece. İkisini aynı tuvale, biri sola biri sağa (`atla`) çizin. Dönüş açısı ile tekrar sayısının çarpımı iki desende de kaç? Bu çarpım 360 olmazsa ne olur, deneyin.

> **Tuvalde:** Solda dört kareden bir yel değirmeni, sağda on iki üçgenden bir çiçek; iki desende de açı × tekrar = 360.

Dizin: `alistirma-koco/kod/KI-1-A1/` — beklenen çıktı yok, çizim

ALIŞTIRMA KI-1-A2 Sayaçlı yineleme, sayarak

Kitapçığın `yineleDizinli(10) { k => kare(20 + 15 * k) }` satırı. Beş kare çizin ama her karenin kenarını da çıktı penceresine yazın: `satıryaz(s"$k: $kenar")`. k 0'dan mı başlıyor, 1'den mi? Son karenin kenarı kaç? Beklenen çıktıyla karşılaştırın; çizim de tuvalde.

çıktı — BEKLENEN-CİKTİ

```
0: 20
1: 35
2: 50
3: 65
4: 80
```

> **Tuvalde:** Aynı köşeden başlayan, kenarları 20, 35, 50, 65, 80 olan beş iç içe kare.

Dizin: `alistirma-koco/kod/KI-1-A2/`

ZORLAYICI KI-1-Z1 Üç sarmal

Kare sarmal: her adımda biraz daha uzun yürü, 90 derece dön: `yineleDizinli(20) { k => ileri(8 * (k + 1)); sağ() }`. Sonra aynı kodu `sağ(120)` ile (üçgen sarmal) ve `sağ(60)` ile (altıgen sarmal) çizin, üçünü yan yana koyun. Hangisi daha sık sarıyor, neden? Bir de `sağ(89)` deneyin: kare neden "dönmeye" başlıyor?

> **DÜŞÜNCE**
>
> Üç sarmal, tek fark dönüş açısı; açıyı işleve girdi yapın.

> **YAKLAŞIM**
>
> `tanım sarmal(açı: Kesir) = yineleDizinli(20) { k => ileri(8 * (k + 1)); sağ(açı) }`; üç çağrı arasında `atla` ile yer değiştirin.

> **İLK SATIR**
>
> `tanım sarmal(açı: Kesir) = { yineleDizinli(20) { k => ileri(8 * (k + 1)); sağ(açı) } }`

> **Tuvalde:** Soldan sağa kare, üçgen ve altıgen sarmallar; açı küçüldükçe sarmal daha çok kol (6) ve daha yavaş açılan bir biçim alır. 89 derece: her turda bir derece kayan, dönen bir kare.

Dizin: `alistirma-koco/kod/KI-1-Z1/` — beklenen çıktı yok, çizim

KEŞİF KI-1-K1 Kâğıttan koda

Kitapçığın “Takım işi”: biriniz kâğıda makul bir desen çizsin (iç içe üçgenler, bir merdiven, bir yel değirmeni, bir ızgara), öteki onu `tanım`'larla koda çevirsin; sonra roller değişsin. Kural: her tekrar eden parça bir `tanım` olsun, hiçbir sayı iki kere yazılmasın. Bu dizin sizin; `ana.kojo`'ya iki desenin kodunu koyun.

DÜŞÜNCE

Kâğıttaki her tekrar bir `tanım`; sayılar bir kere yazılır, gerisi çağrı.

YAKLAŞIM

Önce en küçük parçayı tanımlayın (bir üçgen, bir basamak), sonra onu `yinele` ile çoğaltan ikinci bir tanım; `atla` ile yer değiştirin.

İLK SATIR

```
tanım basamak(boy: Sayı) = { ileri(boy); sağ(); ileri(boy); sol() }
```

Tuvalde: Sizin deseniniz.

Dizin: `alistirma-koco/kod/KI-1-K1/` — beklenen çıktı yok, çizim

I.2

Değerler ve türler; dez / den

`dez` ad takar ve bir daha değişmez; `den` mecbur kalınca. Türler çoğu zaman yazılmaz ama hep vardır: Sayı, Kesir, Yazı, Harf, İkil, Uzun, İriSayı. Ve kardeş kitapçıktan tanıdık bir hata: taşma. Bu konunun alıştırmaları çıktı penceresine yazıyor.

```
1
2
1
```

YANIT YERİNE

kitapçık I.3: Değerler ve türler Skala kılavuzu: veri türleri temel-bilgiler 4: kavramsal makine C++ cildi I.2: taşma

ISINMA KI-2-I1

Yazı kalıbı

Kitapçığın “değerler” örneği olduğu gibi. `kenar`'ı 100 yapın ve çalıştırın: üç satır nasıl değişiyor? Sonra son satırdaki yazı kalıbına üçüncü bir değer ekleyin: `${kenar * boşluk}`. Kıvrık parantez ne zaman gerekiyor, ne zaman `$ad` yetiyor?

ana.kojo — BAŞLANGIÇ

```
// KI-2-I1 · Yazı kalıbı: kenar = 100, sonra kalıba çarpımı ekleyin
dez kenar = 80
dez boşluk = kenar / 4
dez ad = "kelebek"
satıryaz(ad)
satıryaz(kenar + boşluk)
satıryaz(s"kenar $kenar, boşluk $boşluk")
```

çıktı — BEKLENEN-ÇİKTİ

```
kelebek
125
kenar 100, boşluk 25, çarpım 2500
```

Dizin: `alistirma-koco/kod/KI-2-I1/`

ALIŞTIRMA KI-2-A1

Taşma avı

Kitapçıkta 2000'in küpü eksi çıkıyor: Sayı 32 bitlik bir kutu. Peki küpü Sayı'ya sığan en büyük n kaç? 1289, 1290, 1291, 1292 için küpü hem `Sayı` hem `Uzun` olarak yazdırın (`dez u: Uzun = n` deyince çarpım Uzun'da yapılır). Hangi satırda iki sütun ayrışıyor? Sayı'nın sınırı `Sayı.EnİrisiSayı.EnİrisiSayı.EnİrisiS

ALIŞTIRMA KI-2-A2

Hangi tür?

Her satır bir soru: `7 / 2` kaç, `7.0 / 2` kaç, `7 % 2` kaç? `"kelebek".boyu`? `'B'` bir Harf; `5 > 3` bir İkil. Önce kâğıda tahminlerinizi yazın, sonra programı tamamlayıp çalıştırın. İkil'i doğrudan yazdırmak yerine `eğer (5 > 3) "evet" yoksa "hayır"` ile yazdırın (iKojo İngilizce `true` basar, Koco `doğru`; biz ikisine de karışmayalım).

çıktı — BEKLENEN-CİKTİ

```
3
3.5
1
7
B
evet
```

Dizin: `alistirma-koco/kod/KI-2-A2/`

ZORLAYICI KI-2-Z1

Değişmezi değiştirmek

Program bilerek derlenmiyor: `dez` ile tanımlanan `sayaç`'a yeni değer veriliyor. Hata iletisini okuyun (iKojo altta gösterir; “reassignment to val” arayın). Düzeltmenin iki yolu var: `den` yapmak ya da yeni bir `dez` tanımlamak. İkisini de deneyin; hangisi kitapçığın “önce dez” kuralına uyuyor? Sonra son iki satır: Sayı'nın en büyüğüne 1 eklenince ne olur?

DÜŞÜNCE

`dez` bir kere bağlanır; “yeniden atama” iletisi tam bunu söylüyor.

YAKLAŞIM

Ya `den sayaç` yapın ya da `dez yeniSayaç = sayaç + 1` deyin; kitapçığın kuralı ikincisini yeğler. Son satırlar: en büyük `Sayı`'ya 1 eklenince eksiye sarar.

İLK SATIR

```
dez yeniSayaç = sayaç + 1
```

ana.kojo — BAŞLANGIÇ

```
// KI-2-Z1 · Değişmezi değiştirmek (bilerek derlenmiyor)
// Çıktı: 11, 2147483647, -2147483648
dez sayaç = 10
sayaç = sayaç + 1
satıryaz(sayaç)
dez enBüyük = Sayı.EnİrisI
satıryaz(enBüyük)
satıryaz(enBüyük + 1)
```

çıktı — BEKLENEN-ÇİKTİ

```
11
2147483647
-2147483648
```

Dizin: `alistirma-koco/kod/KI-2-Z1/`

I.3

Çıktısı olan işlevler; ilk Project Euler

`tanım topla(a: Sayı, b: Sayı) = a + b`: gövde bir deyiş, değeri neyse çıktı o. `eğer` de bir deyiş. Ve ilk Project Euler çözümü, döngüsüz: `(1 |- 1000).ele(...).topla`. Bu konunun beş alıştırması da sayı veriyor; beklenen çıktılarla karşılaştırın, sonra Project Euler'e girin.

1 2 1 1

> **YANIT YERİNE**
>
> kitapçık I.4: Çıktısı olan işlevler temel-bilgiler 3: bilişim temelleri (işlev) ders 5: PE 1, C++'ta Project Euler

---

ISINMA KI-3-I1 İki işlev daha

Kitapçığın `topla`, `bölünüyorMu` ve `işaret`'i hazır. İki işlev ekleyin: `mutlak(s: Sayı)` (eksiyse ters çevir) ve `enİri3(a, b, c)` (üç sayının en büyüğü; `enİri2`'yi iki kere kullanın). Beşini de çağırıp yazdırın. İkil döndüren `bölünüyorMu`'yu yine `eğer` ile "bölünür/bölünmez"e çevirin.

ana.kojo — BAŞLANGIÇ

```
// KI-3-I1 · İki işlev daha: mutlak ve enİri3
// Çıktı: 8, bölünür, bölünmez, eksi, 5, 9  (ayrı satırlarda)
tanım topla(a: Sayı, b: Sayı) = a + b
tanım bölünüyorMu(sayı: Sayı, bölen: Sayı) = sayı % bölen == 0
tanım enİri2(a: Sayı, b: Sayı) = eğer (a > b) a yoksa b
tanım işaret(s: Sayı) =
  eğer (s > 0) "artı"
  yoksa eğer (s < 0) "eksi"
  yoksa "sıfır"
satıryaz(topla(3, 5))
satıryaz(eğer (bölünüyorMu(15, 5)) "bölünür" yoksa "bölünmez")
satıryaz(eğer (bölünüyorMu(16, 5)) "bölünür" yoksa "bölünmez")
satıryaz(işaret(-5))
// mutlak(-5) ve enİri3(4, 9, 7)
```

çıktı — BEKLENEN-CİKTİ

```
8
bölünür
bölünmez
eksi
5
9
```

Dizin: `alistirma-koco/kod/KI-3-I1/`

ALIŞTIRMA KI-3-A1

Project Euler 1, üç kere

Kitapçığın tek deyişlik çözümünü bir işleve koyun: `pe1(sınır: Sayı)`. Üç kere çağırın: 1000 (233168 çıkmalı), 10 000, ve 1000 ama bu sefer 3, 5 *ya da 7*'ye bölünenler. `|-` ile `|-|` farkına dikkat: sınır dahil mi?

çıktı — BEKLENEN-CİKTİ

```
233168
23331668
271066
```

Dizin: `alistirma-koco/kod/KI-3-A1/`

ALIŞTIRMA KI-3-A2

Project Euler 6

PE 6: ilk yüz sayının toplamının karesi ile karelerinin toplamı arasındaki fark. İki zincir yeter: `(1 |-| 100).topla` ve `(1 |-| 100).işle(s => s * s).topla`. Yanıt 25164150. Sonra 1000 için deneyin: Sayı'ya sığıyor mu? (Toplamın karesi $2{,}5 \cdot 10^{11}$.) Hangi türe geçmeli?

çıktı — BEKLENEN-CİKTİ

```
25164150
250166416500
```

Dizin: `alistirma-koco/kod/KI-3-A2/`

ZORLAYICI İKİLİ KI-3-Z1

Klasik tuzağı ölçün

Kitapçığın “klasik tuzak”ı: 3'e bölünenler + 5'e bölünenler, 15'e bölünenleri iki kere sayar. Tuzağı bilerek kurun: iki ayrı toplamı toplayın, doğru yanıtla farkını alın. (Toplama `yanlış` adını veremezsiniz: Koco'da anahtar sözcük, tıpkı `doğru` gibi; `yanlışToplam` deyin.) Fark neye eşit olmalı? Üçüncü bir zincirle onu da hesaplayıp eşitliği gösterin. 1000 için üç sayı.

DÜŞÜNCE

Yanlış toplam 15'in katlarını iki kere sayar; fark tam olarak o katların toplamı.

YAKLAŞIM

Dört zincir: 3'e bölünenler, 5'e bölünenler, `||` ile doğru toplam, `% 15` ile on beşler. `yanlışToplam = üçler + beşler`; fark ile on beşler eşit çıkmalı.

İLK SATIR

```
dez onBeşler = (1 |- 1000).ele(s => s % 15 == 0).topla
```

çıktı — BEKLENEN-CİKTİ

```
yanlış: 266333
doğru: 233168
fark: 33165
15'e bölünenler: 33165
```

Dizin: `alistirma-koco/kod/KI-3-Z1/`

KEŞİF KI-3-K1

Project Euler 9

PE 9: a < b < c, a + b + c = 1000 ve a² + b² = c² olan tek üçlü; çarpımı isteniyor. İki iç içe aralık: a için 1'den 1000'e, b için a'dan 1000'e; c = 1000 − a − b. Scala'nın `için ... ver` yazımı tam buna göre (Skala kılavuzunda); `eğer` ile süzün. Çarpımı yazdırın; sonra Project Euler'de hesap açıp 1, 6 ve 9'u gönderin.

DÜŞÜNCE

Üçlü tek: a + b + c = 1000 ve a² + b² = c²; iki değişkeni gezin, üçüncüsü hesaplanır.

YAKLAŞIM

`için` yazımı hazır: `a` 1'den 1000'e, `b` `a`'dan 1000'e, `c = 1000 - a - b`, `eğer` ile süzün; ilk ögeyi `başı` ile alın.

İLK SATIR

```
dez (a, b, c) = üçlüler.başı
```

çıktı — BEKLENEN-ÇİKTİ

```
a=200 b=375 c=425 çarpım 31875000
```

Dizin: `alistirma-koco/kod/KI-3-K1/`

I.4

İlk oyununuz

Kitapçığın tahmin oyunu: `rastgele`, `sayıOku`, `yineleDoğruysa` ve mecbur kalınan üç `den`. Asıl ders oyunun arkasındaki hesap: yedi hak neden yeter? İkili arama. Bu konunun alıştırmaları o hesabı makineye yaptırıyor; biri de oyunu tersine çeviriyor.

1 1 2 1

YANIT YERİNE

kitapçık I.5: İlk oyununuz temel-bilgiler 7: oyun tarifleri C++ cildi I.5: tahmin oyunu

ISINMA KI-4-I1

Kaç hak yeter?

Kitapçığın “Deneyin”i: sınır 1000 olsa kaç hak gerekir? Önce `yineleDoğruysa` ile: 1'den başlayıp sınırı geçene kadar ikiye katlayın, kaç katlama yaptığınızı sayın. `hak(sınır: Sayı)` işlevini yazın; 100, 1000 ve 1 000 000 için yazdırın. Sonra `log2tabanlı` ile karşılaştırın: hangisi tam sayı veriyor?

çıktı — BEKLENEN-CİKTİ

```
100 için 7 hak
1000 için 10 hak
1000000 için 20 hak
```

Dizin: `alistirma-koco/kod/KI-4-I1/`

ALIŞTIRMA KI-4-A1

Bilgisayar bulsun

Oyunu tersine çevirmeden önce bilgisayara kendi kendine oynatın: tutulan sayı 73 olsun (şimdilik rastgele değil, sınanabilsin diye). Program her turda aralığın ortasını söylesin, kendi tahminini 73'le karşılaştırıp “büyük” ya da “küçük” desin ve aralığı daraltsın. Her tahmini yazdırın; kaç adımda buldu? Beklenen çıktıyla aynı adımları atmalısınız: orta = (altSınır + üstSınır) / 2.

çıktı — BEKLENEN-CİKTİ

```
50 büyük
75 küçük
62 büyük
68 büyük
71 büyük
73 bildim, 6 adım
```

Dizin: `alistirma-koco/kod/KI-4-A1/`

ZORLAYICI KI-4-Z1

En kötü durum

Yedi hak *her* sayı için yeter mi? KI-4-A1'in aramasını bir işleve koyun: `adımSayısı(tutulan: Sayı, sınır: Sayı): Sayı`, yazdırmadan yalnız adım saysın. Sonra 1'den 100'e bütün sayılar için çağırıp en büyüğünü bulun (`(1 |-| 100).işle(...).enİrisi`). 1000 ve 1 000 000 için de. KI-4-I1'in hesabıyla uyuşuyor mu?

DÜŞÜNCE

Arama yazdırmasın, yalnız saysın; sonra bütün sayılar için en büyük adım.

YAKLAŞIM

`adımSayısı` içinde KI-4-A1'in döngüsü, `satıryaz` satırları silinmiş; dışarıda `(1 |-| sınır).işle(t => adımSayısı(t, sınır)).enİrisi`.

İLK SATIR

`yineleDoğruysa(altSınır <= üstSınır) { dez orta = (altSınır + üstSınır) / 2; adım = adım + 1; ... }`

çıktı — BEKLENEN-CİKTİ

```
100: en çok 7 adım
1000: en çok 10 adım
1000000: en çok 20 adım
```

Dizin: `alistirma-koco/kod/KI-4-Z1/`

ZORLAYICI İKİLİ KI-4-Z2

Tahmini tersine çevirin

Kitapçığın beşinci alıştırması: siz tutun, bilgisayar bulsun. Program ortayı söylesin, siz `satıroku` ile `b` (daha büyük), `k` (daha küçük) ya da `=` girin. KI-4-A1'in döngüsü, yalnız karşılaştırma yerine sizin yanıtınız. Sınamak için `girdi` dosyası hazır: 73'ü tutan bir oyuncunun yanıtları. iKojo'da ise gerçekten oynayın; yanlış yanıt verirseniz program ne yapıyor?

DÜŞÜNCE

Bilgisayar ortayı sorar, siz yön verirsiniz; aralık her yanıtta yarıya iner.

YAKLAŞIM

`yanıt == "="` bulundu; `"b"` ise `altSınır = orta + 1`; `"k"` ise `üstSınır = orta - 1`. Girdi dosyası 73'ü tutan oyuncunun yanıtları.

İLK SATIR

```
eğer (yanıt == "=") { bulundu = doğru } yoksa eğer (yanıt == "b") altSınır = orta + 1 yoksa üstSınır = orta - 1
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: B K B B B =

```
50?
75?
62?
68?
71?
73?
73 bildim, 6 adım
```

Dizin: `alistirma-koco/kod/KI-4-Z2/`

KEŞİF KI-4-K1

Project Euler 2 ve 3

Kitapçığın üçüncü ve dördüncü alıştırmaları, bu sefer kodla. PE 2: dört milyonu aşmayan çift Fibonaççi sayılarının toplamı; `den` ve `yineleDoğruysa` ile (Bölüm IV'te bambaşka yazacağız). PE 3: 600851475143'ün en büyük asal çarpanı; sayı Sayı'ya sığmaz, `Uzun` gerekir (sabitin sonuna `L`). En küçük bölenden başlayıp bölebildikçe bölün; kalan son sayı yanıt. İki sayıyı yazdırın, PE'ye gönderin.

DÜŞÜNCE

PE 2 iki değişkenli döngü; PE 3 en küçük bölenden başlayıp bölerek küçültme.

YAKLAŞIM

PE 2: `yineleDoğruysa(b <= 4000000)`, çiftse topla, `(a, b) = (b, a + b)` için geçici değişken. PE 3: `den b = 2L`, `n % b == 0` iken böl, değilse `b` artır; kalan `n` yanıt.

İLK SATIR

```
yineleDoğruysa(b * b <= n) { eğer (n % b == 0) n = n / b yoksa b = b + 1 }
```

çıktı — BEKLENEN-ÇİKTİ

```
PE2: 4613732
PE3: 6857
```

Dizin: `alistirma-koco/kod/KI-4-K1/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`KI-3-Z1` Klasik tuzağı ölçün	`KI-4-Z1` En kötü durum	`KI-1-Z1` Üç sarmal

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM II

Veriyi Düzenlemek

Kitapçığın ikinci bölümü dizin, küme ve eşlemi, üstlerinde çalışan üç sihirli işlemi (işle, ele, indirge) ve Belki'yi anlatıyor. Burada dört konu, on dokuz alıştırma; hepsi çıktı penceresine yazıyor. Kapları `yazıYap` ile yazdırıyoruz ki beklenen çıktı iKojo'da da Koco'da da aynı olsun. Amaç bir refleks: döngü yazmadan önce bir dur, bu bir zincir mi?

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Soyutlama · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

II.1

Dizin: işlevselin atardamarı

Dizin bir kere kuruldu mu değişmez: `tersi` ve `sıralı` yeni dizin verir, eskisi yerinde durur. Baş, kuyruk ve `::` ile öne ekleme. Dört alıştırma; kitapçığın ilk büyük fikrini elle tutmak için.

1 2 1

YANIT YERİNE

kitapçık II.1: Dizin Skala kılavuzu: kaplar C++ cildi II.1: dizi

ISINMA KII-1-I1

Baş, kuyruk, son

Kitapçığın dizin örneği, kapları `yazıYap(" ")` ile yazdıran hâliyle. Çalıştırın; sonra iki satır ekleyin: `sayılar.sonu` ve `Boş.boyu`. Son satır `sayılar`'ın hâlâ aynı olduğunu gösteriyor: hangi satırlar onu değiştirebilirdi de değiştirmedi?

ana.kojo — BAŞLANGIÇ

```
// KII-1-I1 · Baş, kuyruk, son
dez sayılar = Dizin(3, 1, 4, 1, 5, 9)
satıryaz(sayılar.başı)
satıryaz(sayılar.kuyruğu.yazıYap(" "))
satıryaz(sayılar.boyu)
satıryaz(sayılar.tersi.yazıYap(" "))
satıryaz(sayılar.sıralı.yazıYap(" "))
satıryaz(sayılar.yazıYap(" "))      // hâlâ aynı!
// sonu ve Boş.boyu
```

çıktı — BEKLENEN-ÇİKTİ

```
3
1 4 1 5 9
6
9 5 1 4 1 3
1 1 3 4 5 9
3 1 4 1 5 9
9
0
```

Dizin: `alistirma-koco/kod/KII-1-I1/`

ALIŞTIRMA KII-1-A1

İlk üçün toplamı

Yalnız `başı` ve `kuyruğu` ile bir dizinin ilk üç ögesinin toplamını veren `ilkÜç(d: Dizin[Sayı])` yazın (kuyruğun kuyruğunun başı…). Sonra aynı şeyi `al(3).topla` ile yazıp iki sonucu yan yana yazdırın. Dizin üç ögeden kısaysa birinci sürüm ne yapar? (Deneyin: `Dizin(1, 2)`; hata iletisini okuyun, sonra satırı silin.)

çıktı — BEKLENEN-ÇİKTİ

```
başla: 8, alla: 8
başla: 60, alla: 60
```

Dizin: `alistirma-koco/kod/KII-1-A1/`

ALIŞTIRMA KII-1-A2

Hiçbir şey değişmedi

`dez a = Dizin(3, 1, 2)`; `a.sıralı`, `a.tersi` ve `0 :: a` üç yeni dizin. Üçünü de ada bağlayıp yazdırın, en sonda yine `a`'yı. Beş satır; beşincisi birinciyle aynı olmalı. Sonra C++ cildini hatırlayın: `sort(v.begin(), v.end())` oradaki diziyi yerinde değiştiriyordu. Hangisini daha güvenli buluyorsunuz, neden?

çıktı — BEKLENEN-CİKTİ

```
a: 3 1 2
sıralı: 1 2 3
tersi: 2 1 3
öne 0: 0 3 1 2
a: 3 1 2
```

Dizin: `alistirma-koco/kod/KII-1-A2/`

ZORLAYICI KII-1-Z1

Dizini öne ekleyerek kurun

`Dizin(1, 2, 3, 4, 5)`'i `Dizin(...)` yazmadan, yalnız `::` ve `yineleDizinli` ile kurun: `den d: Dizin[Sayı] = Boş` ile başlayın, her adımda öne bir sayı ekleyin. Öne eklediğiniz için sıra ters çıkar; 1 2 3 4 5 çıkması için hangi sayıyı eklemelisiniz? İki sürümü de yazdırın: önce ters çıkan, sonra düzelttiğiniz. `den` burada mecburi mi? (Bölüm IV'te özyinelemeyle `den`'siz yazacağız.)

DÜŞÜNCE

`::` öne ekler; sona 5 koymak için ilk önce 5'i eklemelisiniz.

YAKLAŞIM

Düz sürümde `k + 1` yerine `5 - k` ekleyin; ya da ters çıkanı `tersi` ile çevirin. `den` burada mecburi, çünkü dizin yeniden bağlanıyor.

İLK SATIR

```
yineleDizinli(5) { k => d = (5 - k) :: d }
```

çıktı — BEKLENEN-CİKTİ

```
ters: 5 4 3 2 1
düz: 1 2 3 4 5
```

Dizin: `alistirma-koco/kod/KII-1-Z1/`

Üç sihirli işlem: işle, ele, indirge

Her ögeye aynı işlevi uygula; soruya doğru yanıt verenleri tut; ikişer ikişer birleştirip tek değere in. Kısaltmaları `topla`, `çarp`, `enİrisi`, `enUfağı`. Asıl güç zincirlemede; bu konunun altı alıştırması zincir kurduruyor.

1 2 2 1

> **YANIT YERİNE**
>
> kitapçık II.2: Üç sihirli işlem Skala kılavuzu: Dizi (herbiriİçin, işle, ele) C++ cildi II: döngülü karşılıkları

ISINMA KII-2-I1 Üçlü, bir de zincir

Kitapçığın `üçlü.kojo`'su: işle, ele, indirge ve dört kısaltma. Çalıştırın, sonra kitapçığın zincirini ekleyin: 100'e kadar sayıların karelerinden 3'e bölünenlerin toplamı. İpucu: kitapçık sonucu yazmıyor; önce tahmin edin (3'e bölünen 33 sayının kareleri...), sonra çalıştırın.

ana.kojo — BAŞLANGIÇ

```
// KII-2-I1 · Üçlü, bir de zincir
dez sayılar = Dizin(3, 1, 4, 1, 5, 9, 2, 6)
satıryaz(sayılar.işle(s => s * s).yazıYap(" "))
satıryaz(sayılar.ele(s => s % 2 == 0).yazıYap(" "))
satıryaz(sayılar.indirge((a, b) => a + b))
satıryaz(sayılar.topla)
satıryaz(sayılar.çarp)
satıryaz(sayılar.enİrisi)
satıryaz(sayılar.enUfağı)
// zincir: (1 |-| 100) kareler, 3'e bölünenler, toplam
```

çıktı — BEKLENEN-CİKTİ

```
9 1 16 1 25 81 4 36
4 2 6
31
31
6480
9
1
112761
```

Dizin: `alistirma-koco/kod/KII-2-I1/`

ALIŞTIRMA KII-2-A1

Takım işi zincirleri

Kitapçığın “Takım işi”nden üç görev, hepsi `sayılar` üstünde: çift sayıların karelerinin en büyüğü; tek sayıların toplamı; 1'den 20'ye 3'e bölünenlerin *kaç tane* olduğu (`say` hazır: `say(s => ...)`). Üç zincir, üç satır. Döngü yok.

çıktı — BEKLENEN-CİKTİ

```
çift karelerin en büyüğü: 36
teklerin toplamı: 19
3'e bölünen: 6 tane
```

Dizin: `alistirma-koco/kod/KII-2-A1/`

ALIŞTIRMA KII-2-A2

Dört kısaltmayı indirge ile

`topla` , `çarp` , `enİrisi` , `enUfağı` birer kısaltma. Dördünü de yalnız `indirge` ile yazın ve kısaltmalarla aynı sonucu verdiklerini yazdırın (`a.enİrisi(b)` iki sayının büyüğünü verir; `eğer` de olur). Sonra düşünün: `indirge` boş dizinde ne yapar? `Boş.topla` ne veriyor; `Dizin[Sayı]().indirge(...)` ne veriyor? İkincisini deneyin, hatayı görün, satırı silin.

çıktı — BEKLENEN-CİKTİ

```
toplam: 31 31
çarpım: 6480 6480
en büyük: 9 9
en küçük: 1 1
boşun toplamı: 0
```

Dizin: `alistirma-koco/kod/KII-2-A2/`

ZORLAYICI İKİLİ KII-2-Z1

Sıra önemli mi?

Kitapçığın zinciri “önce kare al, sonra 3'e bölünenleri ele”. Sırayı değiştirin: önce 3'e bölünenleri ele, sonra karelerini al. Aynı sonuç mu? Şimdi 3 yerine 4 ile ikisini de yazın: bu sefer farklı! Neden? (Hangi sayıların karesi 4'e bölünür?) Dört sayıyı yazdırın; sonra bir cümleyle: işle ile ele'nin sırası ne zaman önemlidir?

DÜŞÜNCE

İşle sonra ele, ele sonra işle: 3 için aynı, 4 için farklı.

YAKLAŞIM

Dört zincir yazın; 4 için karesi 4'e bölünen sayılar çift sayılardır, kendisi 4'e bölünenler değil. Fark buradan.

İLK SATIR

```
dez önceEle4 = (1 |-| 100).ele(s => s % 4 == 0).işle(s => s * s).topla
```

çıktı — BEKLENEN-CİKTİ

```
3: 112761 112761
4: 171700 88400
```

Dizin: `alistirma-koco/kod/KII-2-Z1/`

ZORLAYICI KII-2-Z2

Collatz, zincirle

C++ cildinin Collatz'ı burada da: n çiftse yarısı, tekse 3n + 1; 1'e kaç adımda iner? `adım(n)` işlevini `den` ve `yineleDoğruysa` ile yazın (içeride döngü serbest). Sonra zincir: 1'den 30'a kadar hangi sayı en çok adım atıyor? İpucu: `işle(n => (adım(n), n))` demetler verir, `enİrisi` demetleri soldan karşılaştırır. Yanıt bir satır.

DÜŞÜNCE

Adım sayısı bir döngü; en uzun zincir bir demet dizininin en büyüğü.

YAKLAŞIM

`adım` içinde `yineleDoğruysa(s != 1)`; dışarıda `(1 |-| 30).işle(n => (adım(n), n)).enİrisi` demeti soldan karşılaştırır.

İLK SATIR

```
dez (k, n) = (1 |-| 30).işle(n => (adım(n), n)).enİrisi
```

çıktı — BEKLENEN-CİKTİ

```
en uzun: 27 (111 adım)
```

Dizin: `alistirma-koco/kod/KII-2-Z2/`

KEŞİF KII-2-K1

Project Euler 4

PE 4: iki üç basamaklı sayının çarpımı olan en büyük palindrom. Palindrom sınaması tek satır: sayının yazısı tersine eşit mi (`yazıya` ve `tersi`). Çarpımlar için `için ... ver` (KI-3-K1) ya da iç içe `düzİşle` / `işle` . Yanıtı yazdırın; PE'ye gönderin. Kaç çarpım sınadınız, hesaplayın: 900 × 900'ü yarıya indirmenin yolu var mı?

DÜŞÜNCE

Bütün üç basamaklı çarpımlar; palindrom olanların en büyüğü.

YAKLAŞIM

`için (a <- 100 |-| 999; b <- a |-| 999) ver a * b` çarpımları verir; `ele(palindromMu).enİrisi` . `b` 'yi `a` 'dan başlatmak yarıya indirir.

İLK SATIR

```
dez çarpımlar = için (a <- 100 |-| 999; b <- a |-| 999) ver a * b
```

çıktı — BEKLENEN-CİKTİ

```
906609
```

Dizin: `alistirma-koco/kod/KII-2-K1/`

II.3

Küme ve eşlem; harf saymak döngüsüz

Küme tekrarsız ve sırasız; üstelik bir işlev: `k(4)` “4 kümede mi?” demek. Eşlem bir şeyi başka bir şeye bağlar; olmayan anahtar için dürüst yanıt `Belki` . Kitapçığın harf sayacı `öbekle` ile döngüsüz. Beş alıştırma: kitapçığın bölüm sonu alıştırmaları, teker teker.

1 2 1 1

YANIT YERİNE

kitapçık II.3: Küme ve eşlem Skala kılavuzu: Eşlek, Küme, Belki hızlı küme işlemleri ders 21: küme ve eşlem (C++) C++ cildi II.3: harf sayacı

ISINMA KII-3-I1 Harf sayacı, başka sözcük

Kitapçığın `harfsay.kojo`'su olduğu gibi. Çalıştırıp çıktıyı kitapçıkla karşılaştırın; sonra yazıyı `"programlama dili"` yapın. Hangi harf en çok? Çıktı neden alfabetik; sıralamayı hangi satır yapıyor, silerseniz ne olur (birkaç kere çalıştırın)?

ana.kojo — BAŞLANGIÇ

```
// KII-3-I1 · Harf sayacı, başka sözcük
dez harfler = "kelebek kanadi".ele(h => h != ' ')
dez öbekler = harfler.öbekle(h => h)
dez sayaç = öbekler.işle { durum (harf, öbek) => (harf, öbek.boyu) }
sayaç.dizine.sıralı.herbiriİçin { durum (harf, kaç) =>
  satıryaz(s"$harf -> $kaç")
}
```

çıktı — BEKLENEN-CİKTİ

```
a -> 3
d -> 1
g -> 1
i -> 2
l -> 2
m -> 2
o -> 1
p -> 1
r -> 2
```

Dizin: `alistirma-koco/kod/KII-3-I1/`

ALIŞTIRMA KII-3-A1 Sesli harfler

Kitapçığın birinci alıştırması: bir yazıdaki sesli harfleri `say` ile sayın. `"aeıioöuü"`'yü bir kümeye çevirin (`kümeye`) ve kümenin işlev olduğunu kullanın: `yazı.say(sesliler)`. Tek satır. İki yazı için yazdırın. Büyük harfleri de saymak isteseniz ne eklerdiniz?

çıktı — BEKLENEN-CİKTİ

```
kelebek kanadi: 6 sesli
programlama dili: 6 sesli
```

Dizin: `alistirma-koco/kod/KII-3-A1/`

ALIŞTIRMA KII-3-A2

Tekrar var mı?

Kitapçığın ikinci alıştırması, iki çözümle: `yinelemesiz` (tekrarları atılmış dizin) ile `boyu` karşılaştırması; ve `kümeye` ile aynı karşılaştırma. İkisi de tek satır. Üç dizin için evet/hayır yazdırın; iki çözüm hep aynı yanıtı vermeli. Hangisinin daha hızlı olduğunu Bölüm III'te ölçeceğiz.

çıktı — BEKLENEN-CİKTİ

```
3 1 4 1 5: evet evet
1 2 3: hayır hayır
7 7: evet evet
```

Dizin: `alistirma-koco/kod/KII-3-A2/`

ZORLAYICI İKİLİ KII-3-Z1

Kelime sayacı

Kitapçığın dördüncü alıştırması: harf sayacını kelime sayacına çevirin. Yazıyı `böl(' ')` ile kelime dizinine ayırın; gerisi harf sayacıyla aynı: `öbekle`, `işle`, sırala, yazdır. Cümle: `"bir iki bir üç iki bir"`. En sık kelimeyi de ayrıca yazdırın (`enİrisi`'nin işlev alan hâli: `enİrisi(ç => ç._2)`).

DÜŞÜNCE

Harf yerine kelime; gerisi aynı: öbekle, say, sırala.

YAKLAŞIM

`kelimeler.öbekle(k => k).işle { durum (k, ö) => (k, ö.boyu) }.dizine.sıralı`; en sık için `enİrisi(ç => ç._2)`.

İLK SATIR

```
dez sayaç = kelimeler.öbekle(k => k).işle { durum (k, ö) => (k, ö.boyu) }.dizine.sıralı
```

çıktı — BEKLENEN-CİKTİ

```
bir -> 3
iki -> 2
üç -> 1
en sık: bir (3)
```

Dizin: `alistirma-koco/kod/KII-3-Z1/`

KEŞİF KII-3-K1

Ters sözlük

Kitapçığın beşinci alıştırması: plaka eşlemini tersine çevirin, sayıdan şehre. Boş bir `Eşlem[Sayı, Yazı]()` açıp `herbiriİçin` ile `eşEkle` yapmak en kısası; `işle` ile demetleri çevirip `eşleğe` de olur. Sonra `ters(6)`'yı ve olmayan bir plakayı yazdırın: `ters(99)` patlar, `ters.al(99)` `Hiçbiri` verir (`Belki`), `ters.alYoksa(99, "yok")` dürüst ve kısa. Düşündürücü kısım: iki şehir aynı plakayı paylaşsaydı? Ekleyin (`"Deneme" -> 6`), çalıştırın, açıklayın.

DÜŞÜNCE

Eşlemi ters çevirmek: her (şehir, plaka) çiftini (plaka, şehir) olarak yeni eşleme koymak.

YAKLAŞIM

`plaka.herbiriİçin { durum (ş, p) => ters.eşEkle(p -> ş) }`; sonra `ters(6)`, `ters.alYoksa(99, "yok")`. Aynı plaka iki şehirde olunca sonra gelen öncekini ezer.

İLK SATIR

```
plaka.herbiriİçin { durum (şehir, p) => ters.eşEkle(p -> şehir) }
```

çıktı — BEKLENEN-CİKTİ

```
6 -> Ankara
99 -> yok
6 -> Deneme
```

Dizin: `alistirma-koco/kod/KII-3-K1/`

II.4

Sırası önemli kaplar

Yığın (son giren ilk çıkar), Kuyruk (ilk giren ilk çıkar), ÖncelikSırası (en öncelikli önce). Değişen kaplar; işlevsel saflıktan taviz ama doğru işte doğru araç. Bölüm V'te yığını kuyrukla değiştirmek bir geziyi başka bir geziye çevirecek; buradaki dört alıştırma o güne hazırlık.

1 1 1 1

YANIT YERİNE

kitapçık II.4: Sırası önemli kaplar Skala kılavuzu: EsnekDizik, Aralık ders 21: yığın ve kuyruk (C++) C++ cildi II.4

ISINMA KII-4-I1 Hepsini boşaltın

Kitapçığın örneği 3, 1, 4'ü hem yığına hem kuyruğa koyup birer tane alıyor. Siz hepsini alın: `yineleDoğruysa(yığın.tane > 0)` ile yığın boşalana kadar `al()`, kuyruk için `baştanAl()`. İki satırda iki sıra; hangisi hangisi, bakar bakmaz söyleyin. `tepe` (yığın) ve `başı` (kuyruk) almadan bakmanın yolu; onları da deneyin.

ana.kojo — BAŞLANGIÇ

```
// KII-4-I1 · Hepsini boşaltın
// Çıktı: "yığın: 4 1 3" ve "kuyruk: 3 1 4"
dez yığın = Yığın[Sayı]()
dez kuyruk = Kuyruk[Sayı]()
Dizin(3, 1, 4).herbiriİçin { s =>
  yığın.koy(s)
  kuyruk.ekle(s)
}
satıryaz(yığın.al())        // 4 -- son giren
satıryaz(kuyruk.baştanAl()) // 3 -- ilk giren
```

çıktı — BEKLENEN-CİKTİ

```
yığın: 4 1 3
kuyruk: 3 1 4
```

Dizin: `alistirma-koco/kod/KII-4-I1/`

ALIŞTIRMA KII-4-A1 Parantez dengesi

Yığının klasik işi: `"(()())"` dengeli, `"(()"` ve `")("` değil. Yazıyı harf harf gezin (`herbiriİçin`): açılış parantezini yığına koyun, kapanışta yığından bir tane alın; yığın boşken kapanış gelirse ya da sonunda yığın boş kalmazsa dengesiz. `dengeliMi(yazı: Yazı): İkil`; üç yazı için evet/hayır. Üç çeşit parantezle (`[{`) nasıl genişler?

çıktı — BEKLENEN-CİKTİ

```
(()()): evet
((: hayır
)(: hayır
```

Dizin: `alistirma-koco/kod/KII-4-A1/`

ZORLAYICI KII-4-Z1

Öncelik sırası, ters yüz

`ÖncelikSırası` "en öncelikli önce" verir; Scala'da öntanımlı öncelik *en büyük*. Şehirleri (uzaklık, ad) demetleriyle koyup `baştanAl` ile boşaltın: en uzak önce çıkar. Dijkstra en yakını ister (Bölüm V). (Dikkat: öncelik sırasında `tane` yok, `boyu` var.) Numara: uzaklığı eksi işaretiyle koyun (–uzaklık, ad); en büyük eksi = en yakın. İki sırayı da yazdırın. Demetler soldan karşılaştırılır: eşit uzaklıkta ad belirler.

DÜŞÜNCE

Öncelik sırası en büyüğü önce verir; en yakını önce istiyorsanız uzaklığı eksiye çevirin.

YAKLAŞIM

İkinci sıraya `(-u, ad)` koyun; çıkarırken `-u` ile geri çevirin. Demetler soldan karşılaştırıldığı için eşit uzaklıkta ad belirler.

İLK SATIR

```
şehirler.herbiriİçin { durum (u, ad) => sıra2.ekle((-u, ad)) }
```

çıktı — BEKLENEN-CİKTİ

```
en uzak önce: Bolu(7) Sakarya(5) İzmit(2) Gebze(2)
en yakın önce: İzmit(2) Gebze(2) Sakarya(5) Bolu(7)
```

Dizin: `alistirma-koco/kod/KII-4-Z1/`

KEŞİF KII-4-K1 Gişe kuyruğu

Bir gişe, beş müşteri; her biri (ad, işlem süresi) ile sırayla kuyruğa giriyor. Gişe kuyruğun başındakini alır, işlemi bitirir, sonrakine geçer. Her müşterinin *bekleme* süresini (gişeye gelene kadar geçen) ve ortalamayı yazdırın. Sonra aynı veriyi yığınla işleyin (son gelen önce): ortalama bekleme değişir mi, adil mi? Bir adım ötesi: en kısa işlemi önce alan bir gişe (ÖncelikSırası) ortalamayı ne yapar?

DÜŞÜNCE

Bekleme, önündekilerin işlem sürelerinin toplamı.

YAKLAŞIM

Bir `den geçen = 0`; kuyruktan her müşteri çıktığında bekleme `geçen`, sonra `geçen += süre`. Yığınla aynı döngü, yalnız kap değişir.

İLK SATIR

```
yineleDoğruysa(kuyruk.boyu > 0) { dez (ad, süre) = kuyruk.baştanAl(); satıryaz(s"$ad: $geçen dk bekledi"); geçen = geçen + süre }
```

çıktı — BEKLENEN-CİKTİ

```
kuyrukla:
Ayşe: 0 dk bekledi
Berk: 5 dk bekledi
Ceren: 7 dk bekledi
Deniz: 15 dk bekledi
Ece: 16 dk bekledi
ortalama: 8.6 dk
yığınla:
Ece: 0 dk bekledi
Deniz: 4 dk bekledi
Ceren: 5 dk bekledi
Berk: 13 dk bekledi
Ayşe: 15 dk bekledi
ortalama: 7.4 dk
```

Dizin: `alistirma-koco/kod/KII-4-K1/`

II.5

Geri dönüş: taşma ve tahmin oyunu, kaplarla

Unutma eğrisi tam burada çalışır: I. bölümde öğrendiğiniz iki şey, bu bölümün kaplarıyla bir daha. Kısa iki soru; ikisi de yirmi dakikalık.

2

> **YANIT YERİNE**
>
> kitapçık I.3: Her türün bir sınırı var kitapçık I.5: İlk oyununuz kitapçık II.3: Küme ve eşlem

---

ALIŞTIRMA KII-5-A1

Taşma, dizinde

I. bölümün taşması bu sefer bir dizinde: `Dizin(2000000000, 100000000, 50000000)`'in `topla`'sı `Sayı`'ya sığmaz ve sessizce eksiye döner. Önce bunu yazdırıp görün. Sonra aynı sayıları `Uzun` olarak (sonlarına `L`) bir dizine koyup yeniden toplayın. İki satır çıktı: yanlış toplam, doğru toplam. Hangi satır "hata" vermedi ama yanlıştı?

çıktı — BEKLENEN-CİKTİ

```
Sayı ile: -2144967296
Uzun ile: 2150000000
```

Dizin: `alistirma-koco/kod/KII-5-A1/`

ALIŞTIRMA KII-5-A2

Skor tablosu

Tahmin oyununun oyuncuları ve kaç tahminde bildikleri satır satır geliyor: `ada 5`, `can 7` ... son satır `bitti`. `satıroku` ile okuyun, satırı `böl(' ')` ile ikiye ayırın, her oyuncunun **en iyi** (en küçük) sonucunu bir `Eşlem[Yazı, Sayı]`'da tutun (`alYoksa` ile eski değere bakın). Sonunda `dizine.sıralı` ile tabloyu yazdırın: `ada: en iyi 4`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: ADA 5 CAN 7 ADA 4 ECE 6 CAN 9 BİTTİ

```
ada: en iyi 4
can: en iyi 7
ece: en iyi 6
```

Dizin: `alistirma-koco/kod/KII-5-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

- `KII-2-Z2` Collatz, zincirle
- `KII-3-Z1` Kelime sayacı
- `KII-4-Z1` Öncelik sırası, ters yüz

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM III

Soyutlama Sanatı

Tekrara ad takmak, işlevi işleve vermek, kendi yinele'nizi yazmak, programı kendi kendini sınar hâle getirmek ve ölçmeden önce saymak. Dört konu, on sekiz alıştırma. Çizenler kitapçığın çokgen ve dönence'sinden türüyor; sayanlar asallarla uğraşıyor. Süre ölçen iki alıştırmanın beklenen çıktısı yok: kendi sayılarınızı üretin, değişmeyen şey orandır.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Soyutlama · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

III.1

Tekrara ad takın

Üçgen, kare, altıgen: üç işlev, aralarındaki tek fark bir sayı. Kendini tekrar etme: `çokgen(kenarSayısı, kenar)`. Soyutlamanın yarısı iyi ad koymak. Dört alıştırma; ikisi çizip sayıyor, biri hesabı DRY yapıyor.

1 2 1

> YANIT YERİNE
>
> kitapçık III.1: Tekrara ad takın temel-bilgiler 3: soyutlama C++ cildi III: zanaat

ISINMA KIII-1-I1 Çember olmaya çalışan çokgen

Kitapçığın `çokgen`'i: 3, 4, 6 ve 60 kenar. Her çağrıdan önce dış açıyı da yazdırın: `360.0 / kenarSayısı`. Sonra `çokgen(360, 1)` ekleyin: tuvalde ne var? Dış açı neden `360.0` ile hesaplanıyor da `360` ile değil (KI-2-A2'yi hatırlayın)?

ana.kojo — BAŞLANGIÇ

```
// KIII-1-I1 · Çember olmaya çalışan çokgen
// Çıktı: "3 kenar: 120.0" biçiminde satırlar
tanım çokgen(kenarSayısı: Sayı, kenar: Kesir) = {
  yinele(kenarSayısı) {
    ileri(kenar)
    sağ(360.0 / kenarSayısı)
  }
}
çokgen(3, 120)
çokgen(4, 90)
çokgen(6, 60)
çokgen(60, 6)
```

çıktı — BEKLENEN-CİKTİ

```
3 kenar: 120.0
4 kenar: 90.0
6 kenar: 60.0
60 kenar: 6.0
360 kenar: 1.0
```

> **Tuvalde:** İç içe üçgen, kare, altıgen, 60-gen ve 360-gen; son ikisi çember gibi.

Dizin: `alistirma-koco/kod/KIII-1-I1/`

ALIŞTIRMA KIII-1-A1

Yıldıza genelleme

Kitapçığın ikinci alıştırması: `yıldız(köşe: Sayı, kenar: Kesir)`. Beş köşelide dönüş 144'tü; formül ne? (İpucu: kaplumbağa iki tam tur atıyor: 2 × 360 / köşe.) Yedi ve dokuz köşeli için de çalıştırın, dönüş açılarını yazdırın. Sonra "üç tur"lu sürümü deneyin: 3 × 360 / 7 ne çizer?

çıktı — BEKLENEN-CİKTİ

```
5 köşe: 144.0
7 köşe: 102.85714285714286
9 köşe: 80.0
```

> **Tuvalde:** Beş, yedi ve dokuz köşeli yıldızlar; 3 × 360 / 7 ile yedi köşeli ama daha sivri bir yıldız.

Dizin: `alistirma-koco/kod/KIII-1-A1/`

ALIŞTIRMA KIII-1-A2

Hesapta da tekrar etmeyin

Program PE 1'i üç kere aynı zinciri yazarak çözüyor: 3'ün katları, 5'in katları, 15'in katları. Tekrarı bir işleve çıkarın: `katlarınToplamı(k: Sayı, sınır: Sayı)`. Sonra kapsama-dışlama: 3 + 5 − 15. Yanıt yine 233168. Dört satırlık program üç satıra iner; asıl kazanç nedir?

ana.kojo — BAŞLANGIÇ

```
// KIII-1-A2 · Hesapta da tekrar etmeyin
// Çıktı: 233168
dez üçler = (1 |- 1000).ele(s => s % 3 == 0).topla
dez beşler = (1 |- 1000).ele(s => s % 5 == 0).topla
dez onbeşler = (1 |- 1000).ele(s => s % 15 == 0).topla
satıryaz(üçler + beşler - onbeşler)
```

çıktı — BEKLENEN-CİKTİ

```
233168
```

Dizin: `alistirma-koco/kod/KIII-1-A2/`

ZORLAYICI İKİLİ KIII-1-Z1

Izgara, iki yoldan

`ızgara(satır, sütun, kenar)`: kareleri yan yana ve alt alta (`atla` ile) dizen bir işlev. Çizdiğiniz çizgileri sayın: `kare`'nin içine `den çizgi` sayacı koyun. 3×4 ızgara kaç çizgi? Ortak kenarlar iki kere çiziliyor; ızgarayı yalnız uzun yatay ve dikey çizgilerle kuran sürüm kaç `çiz` çağrısıyla aynı resmi çizer? İkisini de yazdırın: önce kare kare, sonra çizgi çizgi (sayaç `ileri`'yi sayan bir `çiz(boy)` işlevine taşınabilir).

DÜŞÜNCE

Kare kare çizince ortak kenarlar iki kere; uzun çizgilerle her çizgi bir kere.

YAKLAŞIM

Birinci sürüm: iki iç içe döngü, her karede `kare`, aralarda `atla`. İkinci sürüm: `satır + 1` yatay ve `sütun + 1` dikey uzun çizgi; sayaç her `çiz`'de artar.

İLK SATIR

```
tanım ızgara(satır: Sayı, sütun: Sayı, kenar: Kesir) = yineleDizinli(satır) { s => yineleDizinli(sütun) { k => atla(k * kenar, s * kenar); kare(kenar) } }
```

çıktı — BEKLENEN-CİKTİ

```
kare kare: 48 çizgi
çizgi çizgi: 9 çizgi
```

Tuvalde: Yan yana iki eş 3×4 ızgara; biri 48, öteki 9 çizgiyle çizilmiş, gözle fark yok.

Dizin: `alistirma-koco/kod/KIII-1-Z1/`

III.2

İşlevi girdi alan işlev

`dönence(12, 30) { çokgen(4, 100) }`: tekrar eden şey bir sayı değil, çizimin kendisi; o da girdi olur. `yinele` de böyle bir işlevmiş, beş satırda kendi sürümünüzü yazıyorsunuz. Altı alıştırma; işlevleri elden ele vermek.

1 2 2 1

YANIT YERİNE

kitapçık III.2: İşlevi girdi alan işlev Skala kılavuzu: adsız işlevler C++ cildi III: işlev işaretçileri yok, kalıplar var

ISINMA KIII-2-I1

Kaç kere çizdi?

Kitapçığın üç `dönence`'si. Bloğun kaç kere çalıştığını sayın: `den sayaç = 0`, her bloğa `sayaç = sayaç + 1`. Üç çağrıdan sonra sayaç kaç olmalı? Önce söyleyin, sonra yazdırın. `çizim: => Birim` girdisi "çalıştırılmamış komutlar": sayacın artması bunu kanıtlıyor mu?

ana.kojo — BAŞLANGIÇ

```
// KIII-2-I1 · Kaç kere çizdi?
// Çıktı: "sayaç: ..."
tanım çokgen(kenarSayısı: Sayı, kenar: Kesir) = {
  yinele(kenarSayısı) {
    ileri(kenar)
    sağ(360.0 / kenarSayısı)
  }
}
tanım dönence(kaç: Sayı, açı: Kesir)(çizim: => Birim) = {
  yinele(kaç) {
    çizim
    sağ(açı)
  }
}
canlandırmaHızınıKur(10)
dönence(12, 30) { çokgen(4, 100) }
dönence(36, 10) { daire(80) }
dönence(8, 45) {
  kalemRenginiKur(rastgeleRenk)
  çokgen(3, 80)
}
```

çıktı — BEKLENEN-ÇİKTİ

```
sayaç: 56
```

Tuvalde: Kitapçığın danteli: kare çiçeği, 36 daire, sekiz renkli üçgen üst üste.

Dizin: `alistirma-koco/kod/KIII-2-I1/`

ALIŞTIRMA KIII-2-A1

bizimYineleDizinli

Kitapçığın “Takım işi”: `bizimYinele` hazır; sayacı da veren `bizimYineleDizinli(kaç: Sayı)(komutlar: Sayı => Birim)`'yi yazın. Özyinelemeli olacak; sayaç 0'dan başlasın ki `yineleDizinli` ile aynı davransın. İkisini yan yana çalıştırıp aynı satırları yazdırın. Sonra onunla bir merdiven çizin: k'inci basamak k + 1 birim.

çıktı — BEKLENEN-CİKTİ

```
0 1 2 3 4
0 1 2 3 4
```

> **Tuvalde:** Her basamağı bir öncekinden uzun sekiz basamaklı bir merdiven.

Dizin: `alistirma-koco/kod/KIII-2-A1/`

ALIŞTIRMA KIII-2-A2

İki kere uygula

İşlev alan iki küçük işlev: `ikiKere(f: Sayı => Sayı, x: Sayı)` f'yi iki kere uygular; `bileşke(f, g)` ise *yeni bir işlev* döndürür: önce g, sonra f. `ikiKere(s => s * 2, 5)`, `ikiKere(s => s + 3, 5)`, `bileşke(s => s * 2, s => s + 3)(5)` ve tersini yazdırın. Son ikisi neden farklı?

çıktı — BEKLENEN-CİKTİ

```
20
11
16
13
```

Dizin: `alistirma-koco/kod/KIII-2-A2/`

ZORLAYICI KIII-2-Z1

Kendi işle'niz, kendi ele'niz

`işle` ve `ele` de sihir değil. `bizimİşle(d: Dizin[Sayı])(f: Sayı => Sayı): Dizin[Sayı]` ve `bizimEle(d)(deneme: Sayı => İkil)` yazın; şimdilik `den`, `herbiriİçin` ve `::` ile (öne ekleyip sonda `tersi`; KII-1-Z1). Gerçek `işle` / `ele` ile aynı sonucu verdiklerini yazdırın. Bölüm IV'te ikisini de `den`'siz, özyinelemeyle yazacağız.

DÜŞÜNCE

`işle` her ögeye `f` uygulayıp yeni dizin kurar; `ele` deneyi geçenleri alır.

YAKLAŞIM

`den sonuç: Dizin[Sayı] = Boş`; `d.herbiriİçin(x => sonuç = f(x) :: sonuç)`; sonda `sonuç.tersi`. `ele` aynı, yalnız `eğer (deneme(x))`.

İLK SATIR

```
tanım bizimİşle(d: Dizin[Sayı])(f: Sayı => Sayı): Dizin[Sayı] = { den sonuç: Dizin[Sayı] = Boş; d.herbiriİçin(x => sonuç = f(x) :: sonuç); sonuç.tersi }
```

çıktı — BEKLENEN-CİKTİ

```
işle: 9 1 16 1 25 = 9 1 16 1 25
ele: 3 1 1 5 = 3 1 1 5
```

Dizin: `alistirma-koco/kod/KIII-2-Z1/`

ZORLAYICI KIII-2-Z2

Sayan döngü

`kaçKere(koşul: => İkil)(adım: => Birim): Sayı`: koşul doğru kaldıkça adımı çalıştırsın ve kaç kere çalıştırdığını döndürsün. İki girdisi de “çalıştırılmamış kod”. Onunla iki eski soruyu yeniden yazın: KI-4-I1'in hak hesabı (1000 için) ve KII-2-Z2'nin Collatz adımı (27 için). Döngü yazmadınız; döngüyü *bir kere* yazıp iki kere kullandınız.

DÜŞÜNCE

Döngüyü bir kere yazıp iki kere kullanmak: koşul ve adım, çalıştırılmamış kod olarak girdi.

YAKLAŞIM

`yineleDoğruysa(koşul) { adım; sayı = sayı + 1 }`; Collatz için `den s = 27` ve `kaçKere(s != 1) { s = eğer (s % 2 == 0) s / 2 yoksa 3 * s + 1 }`.

İLK SATIR

```
yineleDoğruysa(koşul) { adım; sayı = sayı + 1 }
```

çıktı — BEKLENEN-CİKTİ

```
1000 için 10 hak
27 için 111 adım
```

Dizin: `alistirma-koco/kod/KIII-2-Z2/`

KEŞİF KIII-2-K1 Dönence içinde dönence

`dönence`'nin bloğuna başka bir `dönence` koyun: `dönence(6, 60) { dönence(12, 30) { çokgen(3, 40) } }`. Sonra her katmana `rastgeleRenk`, kalem kalınlığı, `zıpla` ile aralık ekleyin. En güzel deseni kim bulur? Bu dizin sizin; sayfa görüntüsünü sınıfla paylaşın. Bir de sayın: iç blok toplam kaç kere çalıştı?

DÜŞÜNCE

Dönence içinde dönence: dıştaki her turda içteki tamamen çalışır.

YAKLAŞIM

İç blok 6 × 12 kere çalışır; sayaç için bir `den`. Renk ve kalınlık için `kalemRenginiKur(rastgeleRenk)` iç bloğun başına.

İLK SATIR

```
dönence(6, 60) { kalemRenginiKur(rastgeleRenk); dönence(12, 30) { çokgen(3, 40) } }
```

Tuvalde: Altı kollu, her kolu on iki üçgenden bir çiçek olan bir desen; iç blok 72 kere çalışır.

Dizin: `alistirma-koco/kod/KIII-2-K1/` — beklenen çıktı yok, çizim

III.3

Kendi kendini sınayan program

`belirt(koşul, mesaj)`: koşul yanlışsa program o an durur. Denemeleri programın içine koyun; kodu bozar bozmaz hangi sözünüzü bozduğunuzu söylesin. İyi deneme kümesi yanlış olması gerekenleri de sınar (91). Dört alıştırma; hepsi önce deneme, sonra kod.

1 2 1

YANIT YERİNE

kitapçık III.3: Kendi kendini sınayan program Skala kılavuzu: İkil, belirt ders 22: assert (C++) C++ cildi III.3: altın dosya

ISINMA KIII-3-I1 İki deneme daha

Kitapçığın `asal.kojo`'su. İki deneme ekleyin: 13 asal, 15 değil. Denemeleri sayan bir `den` koyun ve sonda “6 deneme geçti.” yazdırın (sayan sarmalayıcıya `dene` diyemezsiniz: anahtar sözcük; kitapçık da `denemeler` demişti). Sonra kitapçığın “Deneyin”i: `n > 1 &&`'i silin, çalıştırın; hangi deneme hangi mesajla patlıyor? Geri koyun.

ana.kojo — BAŞLANGIÇ

```
// KIII-3-I1 · İki deneme daha
// Çıktı: "6 deneme geçti."
tanım asalMı(n: Sayı): İkil =
  n > 1 && (2 |- n).hepsiDoğruMu(b => n % b != 0)
tanım denemeler() = {
  belirt(asalMı(2), "2 asaldır")
  belirt(asalMı(97), "97 asaldır")
  belirt(!asalMı(1), "1 asal sayılmaz")
  belirt(!asalMı(91), "91 = 7 × 13, kolay tuzak")
  satıryaz("denemeler geçti.")
}
denemeler()
```

çıktı — BEKLENEN-ÇİKTİ

```
6 deneme geçti.
```

Dizin: `alistirma-koco/kod/KIII-3-I1/`

ALIŞTIRMA KIII-3-A1 Mükemmel sayılar

Kitapçığın üçüncü alıştırması: kendisi dışındaki bölenlerinin toplamı kendisine eşit (6 = 1 + 2 + 3). `mükemmelMi(n)`'yi `ele` ve `topla` ile tek satırda yazın. Önce denemeler: 6 ve 28 mükemmel, 12 ve 1 değil. Sonra 10 000'e kadar olanları bulup yazdırın. Kaç tane? (Beşincisi 33 550 336; bulmaya kalkmayın, neden?)

çıktı — BEKLENEN-ÇİKTİ

```
denemeler geçti.
6 28 496 8128
```

Dizin: `alistirma-koco/kod/KIII-3-A1/`

ALIŞTIRMA KIII-3-A2

Önce denemeler: ebob

İki sayının en büyük ortak böleni; Öklid'in yöntemi: `ebob(a, b) = ebob(b, a % b)`, `b == 0` olunca a. Önce denemeleri yazın: `ebob(12, 18) == 6`, `ebob(7, 5) == 1`, `ebob(0, 5) == 5`, `ebob(1071, 462) == 21`. Sonra işlevi; `yineleDoğruysa` ile ya da özyinelemeyle (Bölüm IV'ün ön tadı). Denemeler geçince `ekok`'u da ekleyin: a / ebob × b.

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
ebob(1071, 462) = 21
ekok(4, 6) = 12
```

Dizin: `alistirma-koco/kod/KIII-3-A2/`

ZORLAYICI İKİLİ KIII-3-Z1

Eşleme denemesi

Kitapçığın dördüncü alıştırması. `asalMıHızlı` karekök sınırıyla yazılmış ama içinde bir hata var: sınır `b * b < n`. Eşleme denemesi onu yakalar: 1'den 1000'e iki sürümün ayrıştığı ilk sayıyı yazdırın (`bul`). Hatayı düzeltin (`<=`), yeniden çalıştırın: ayrışma kalmadı mı? Sonunda 1000'e kadar kaç asal var, yazın. Yavaş ama basit sürüm, hızlı sürümün hakemi.

DÜŞÜNCE

Yavaş sürüm hakem; hızlı sürümle ayrıştığı ilk sayı hatayı gösterir.

YAKLAŞIM

`bul` bir `Belki` verir: `Biri(n)` ya da `Hiçbiri`; `eşle` ile yazdırın. Hata `b * b < n`'de: tam kare sayılarda (25, 49) kök dahil edilmiyor.

İLK SATIR

```
ayrışan eşle { durum Biri(n) => satıryaz(s"ilk ayrışma: $n");
durum Hiçbiri => satıryaz("ayrışma yok") }
```

ana.kojo — BAŞLANGIÇ

```
// KIII-3-Z1 · Eşleme denemesi (hızlı sürümde bilerek hata var)
// Çıktı: "ilk ayrışma: ...", düzeltince "ayrışma yok" ve "1000'e kadar 168 asal"
tanım asalMı(n: Sayı): İkil =
  n > 1 && (2 |- n).hepsiDoğruMu(b => n % b != 0)
tanım asalMıHızlı(n: Sayı): İkil =
  n > 1 && (2 |-| 1000).alDoğruKaldıkça(b => b * b < n).hepsiDoğruMu(b => n % b != 0)
dez ayrışan = (1 |-| 1000).bul(n => asalMı(n) != asalMıHızlı(n))
// ayrışan bir Belki: Biri(n) ya da Hiçbiri; yazdırın
```

çıktı — BEKLENEN-ÇİKTİ

```
ayrışma yok
1000'e kadar 168 asal
```

Dizin: `alistirma-koco/kod/KIII-3-Z1/`

III.4

Tahmin etmeyin, ölçün — ve sayın

Kareköke kadar bakmak yeter: $10^9 + 7$ için otuz bin kat daha az deneme. Bu bir ölçüm değil, sayım; kâğıt üstünde belli. Sonra `buSaniye` ile ölçün. Ölçülen süreler makineden makineye değişir; bu yüzden iki alıştırmanın beklenen çıktısı yok, kendi sayılarınızı üretin. Değişmeyen şey orandır.

1 1 1 1

> **YANIT YERİNE**
>
> kitapçık III.4: Tahmin etmeyin, ölçün hız ölçümü (C++) C++ cildi III.5: hız ölçümü

ISINMA KIII-4-I1

Önce sayın

Kitapçığın tablosunu makineye yazdırın: 10 007, 1 000 003 ve $10^9 + 7$ için “2'den n'ye” kaç deneme (n − 2), “√n'ye kadar” kaç deneme (`karekökü(n).sayıya - 1`). Üç satır; son satırdaki oran kaç? Hiçbir şey ölçmediniz, yalnız saydınız; ama bu sayım hangi sürümün kazanacağını baştan söylüyor.

çıktı — BEKLENEN-CİKTİ

```
10007: 10005 99
1000003: 1000001 999
1000000007: 1000000005 31621
```

Dizin: `alistirma-koco/kod/KIII-4-I1/`

ALIŞTIRMA KIII-4-A1

Şimdi ölçün

Kitapçığın `süresi`'yle iki sürümü aynı sayıda ölçün: `asalMı(999999937)` ve `asalMıHızlı(999999937)` (KIII-3-Z1'in düzeltilmiş hâli). Yavaş sürüm bir süre bekletir; sabırlı olun. İki süreyi ve oranını yazdırın. Beklenen çıktı yok: sizin makinenizin sayıları. Oran kitapçığın otuz bin katına yakın mı, değilse neden?

Dizin: `alistirma-koco/kod/KIII-4-A1/` — beklenen çıktı yok, ölçüm

ZORLAYICI KIII-4-Z1

Tekrar var mı, ölçün

Kitapçığın beşinci alıştırması: KII-3-A2'nin iki çözümünü (`yinelemesiz` ve `kümeye`) 10 bin, 100 bin ve 1 milyon ögelik dizinlerde ölçün. Dizin: `(1 |-| n).dizine.işle(s => s % 1000)` (tekrarlı). Bir tablo yazdırın. Süreler nasıl büyüyor: n iki katına çıkınca süre kaç katına çıkıyor? Beklenen çıktı yok. Bu tablo C++ cildinin karmaşıklık konusunun kapısı.

> **DÜŞÜNCE**
>
> Üç boy, iki yöntem, altı ölçüm; süreleri bir tabloya yazın.

> **YAKLAŞIM**
>
> `süresi` hazır; her `n` için iki satır `satıryaz(s"$n: yinelemesiz ${süresi(tekrarVar1(d))} s, küme ${süresi(tekrarVar2(d))} s")`.

> **İLK SATIR**
>
> ```
> satıryaz(s"$n: ${süresi(tekrarVar1(d))} ${süresi(tekrarVar2(d))}")
> ```

Dizin: `alistirma-koco/kod/KIII-4-Z1/` — beklenen çıktı yok, ölçüm

KEŞİF KIII-4-K1

Asal sayma yarışı

100 000'e kadar kaç asal var? İki yol: her sayıya `asalMıHızlı` (√n'ye kadar deneme) ve Eratosthenes'in eleği: `Dizik.doldur(n + 1)(doğru)` ile bir dizik, her asalın katlarını `yanlış` yapın, kalan `doğru`'ları sayın (`say`). İki sayı eşit çıkmalı: 9592. Sonra ikisini `süresi` ile ölçün; elek kaç kat hızlı? Bir adım ötesi: eleği 10^7'ye çıkarın.

DÜŞÜNCE

İki yöntem aynı sayıyı vermeli; elek katları siler, deneme bölenleri dener.

YAKLAŞIM

Elek: `(2 |-| 316).herbiriİçin { i => eğer (elek(i)) (i * i |-| n).adımla(i).herbiriİçin(k => elek(k) = yanlış) }`; sonra `elek.say(x => x)`. (`adımla` yoksa `yineleDoğruysa` ile.)

İLK SATIR

```
den k = i * i; yineleDoğruysa(k <= n) { elek(k) = yanlış; k = k + i }
```

çıktı — BEKLENEN-CİKTİ

```
deneyerek: 9592
elekle: 9592
```

Dizin: `alistirma-koco/kod/KIII-4-K1/`

III.5

Geri dönüş: eski işlevleri sınamak

Bu bölümün alışkanlığı sınamak; sınanacak şey de önceki bölümlerden. I. bölümün `bölünüyorMu`'su ve II. bölümün harf sayacı, bu sefer `belirt` ile.

2

YANIT YERİNE

kitapçık I.4: Çıktısı olan işlevler kitapçık II.3: Harf saymak, döngüsüz kitapçık III.3: Kendi kendini sınayan program

ALIŞTIRMA KIII-5-A1

bölünüyorMu için denemeler

I. bölümün `bölünüyorMu(a, b)`'sunu yazın ve dört `belirt` ile sınayın: 15 5'e bölünür, 15 4'e bölünmez, 0 her sayıya bölünür, **b sıfırsa yanıt yanlış** (programı çökertmeyin: `15 % 0` çalışırken patlar). Sonra çiftler dizini için `15 5 -> evet` biçiminde yazın.

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
15 5 -> evet
15 4 -> hayır
0 3 -> evet
7 0 -> hayır
```

Dizin: `alistirma-koco/kod/KIII-5-A1/`

ALIŞTIRMA KIII-5-A2

Harf sayacını sınayın

II. bölümün harf sayacını bir işleve koyun: `harfSay(yazı)` boşlukları atsın, `öbekle` ile sayıp `(harf, sayı)` çiftlerinin **sıralı dizinini** döndürsün. `belirt` ile sınayın: `harfSay("aab")` tam olarak `Dizin(('a', 2), ('b', 1))` olmalı; `harfSay("")` boş. Sonra `kelebek kanadi` için sayımı `a: 2` biçiminde yazın.

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
a: 2
b: 1
d: 1
e: 3
i: 1
k: 3
l: 1
n: 1
```

Dizin: `alistirma-koco/kod/KIII-5-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

- `KIII-2-Z2` Sayan döngü
- `KIII-3-Z1` Eşleme denemesi
- `KIII-1-Z1` Izgara, iki yoldan

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM IV

Özyineleme ve Fraktallar

Taban ve adım; sarmal, ağaç, Koch tanesi, Hanoi; dizinlerde eşle/durum ile özyineleme ve kenar defteriyle hızlandırma. Dört konu, on sekiz alıştırma. Çizenler çizerken sayıyor: kaç çizgi, kaç dal, kaç üçgen; sayı kâğıtta hesaplanabiliyor, tuvalde doğrulanıyor.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Soyutlama · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

IV.1

Taban ve adım; bir ağaç çizelim

Her özyinelemeli işlevin iki parçası: kendini çağırmadan durduğu taban, problemi küçülterek kendine indirgediği adım. Sarmal bir kere, ağaç iki kere çağırıyor; `geri(boy)` hayati. Beş alıştırma, çoğu çizip sayıyor.

1 2 1 1

YANIT YERİNE

kitapçık IV.1-2: Taban ve adım, ağaç temel-bilgiler 2: çekirdek fikirler C++ cildi IV.1: özyineleme

ISINMA KIV-1-I1 Sarmalı sayın

Kitapçığın sarmalı: taban `boy > 2`, adım `boy * 0.95`. Kaç çizgi çiziyor? Tahmin edin, sonra bir `den` sayaçla sayıp yazdırın. Sonra kitapçığın deneyleri: tabanı 150 yapın (kaç çizgi?), küçülmeyi kaldırın (ne oluyor, Koco nasıl durduruyor?). Sayaç her deneyde yanıtı veriyor.

ana.kojo — BAŞLANGIÇ

```
// KIV-1-I1 · Sarmalı sayın
// Çıktı: "sarmal(200): ... çizgi"
tanım sarmal(boy: Kesir): Birim =
  eğer (boy > 2) {
    ileri(boy)
    sağ()
    sarmal(boy * 0.95)
  }
canlandırmaHızınıKur(10)
sarmal(200)
```

çıktı — BEKLENEN-CİKTİ

```
sarmal(200): 90 çizgi
```

> **Tuvalde:** İçe kıvrılan kare sarmal; 90 çizgi.

Dizin: `alistirma-koco/kod/KIV-1-I1/`

ALIŞTIRMA KIV-1-A1 Dalları sayın

Kitapçığın ağacı: `ağaç(90)`, oran 0,7, taban 4. Kaç dal çiziliyor? Her çağrı bir dal; sayaçla sayın. Kâğıtta: gövde 90, 63, 44, 31, ... 4'ü geçtiği sürece devam; kaç seviye, her seviyede kaç dal? Sonra kalem kalınlığını `boy / 10` yapın ve açıları değiştirin; sayı değişir mi?

çıktı — BEKLENEN-CİKTİ

```
ağaç(90): 511 dal
```

> **Tuvalde:** Dokuz seviyeli, gövdesi kalın uçları ince bir ağaç; 511 dal.

Dizin: `alistirma-koco/kod/KIV-1-A1/`

ALIŞTIRMA KIV-1-A2

Sayılarda taban ve adım

Çizmeyen üç özyineleme: `toplam(n)` = 1'den n'ye toplam; `faktöriyel(n): Uzun`; `üs(a, n)` = a^n. Her birinde önce tabanı yazın (n == 0 için ne?), sonra adımı. `toplam(100)`, `faktöriyel(20)`, `üs(2, 30)`'u yazdırın. `faktöriyel(21)` neden yanlış çıkar?

çıktı — BEKLENEN-CİKTİ

```
5050
2432902008176640000
1073741824
```

Dizin: `alistirma-koco/kod/KIV-1-A2/`

ZORLAYICI İKİLİ KIV-1-Z1

Sierpinski üçgeni

Kitapçığın üçüncü alıştırması. Taban: derinlik 0'da düz bir üçgen çiz. Adım: üç köşeye yarı boyutlu, bir eksik derinlikli üç üçgen. Köşeler arasında `zıpla` ile gidin; `konumVeYönüBelleğeYaz` / `konumVeYönüGeriYükle` başlangıca dönmeyi kolaylaştırır. Derinlik 4'te kaç dolu üçgen çizildi? Sayaçla yazdırın; 3^4 ile karşılaştırın.

DÜŞÜNCE

Derinlik 0 dolu üçgen; derinlik d, üç köşeye d−1 derinlikli üç üçgen.

YAKLAŞIM

Her köşe için: `sierpinski(kenar / 2, derinlik - 1)`, sonra `zıpla(kenar / 2); sol(120)` ile sıradaki köşeye. Üçgen sayısı 3^d.

İLK SATIR

```
yinele(3) { sierpinski(kenar / 2, derinlik - 1); zıpla(kenar / 2); sol(120) }
```

çıktı — BEKLENEN-CİKTİ

```
derinlik 4: 81 üçgen
```

Tuvalde: Kenarı 320 olan büyük üçgenin içinde 81 küçük üçgen: Sierpinski örgüsü.

Dizin: `alistirma-koco/kod/KIV-1-Z1/`

KEŞİF KIV-1-K1

Eğrelti otu ~2 saat

Kitapçığın birinci alıştırması: ağaca üçüncü bir dal (ortaya, daha kısa) ekleyin; açılarla, oranlarla oynayın; dalın boyuna göre rengi yeşilden kahverengiye değiştirin (kalın dallar kahverengi, ince uçlar yeşil: `eğer (boy > 30) kahverengi yoksa yeşil`). Dal sayısını yine yazdırın; beklenen çıktı yok, sayı sizin oranlarınıza bağlı (bizim ayarlarla 977 çıktı). En güzel ağacı kim çizecek; bu dizin sizin.

DÜŞÜNCE

Üçüncü dal ortaya, daha kısa; renk dalın boyuna bağlı.

YAKLAŞIM

İki dalın arasına `ağaç(boy * 0.5)` ekleyin (yönü değiştirmeden); `ileri`'den önce `kalemRenginiKur(eğer (boy > 30) kahverengi yoksa yeşil)`.

İLK SATIR

```
kalemRenginiKur(eğer (boy > 30) kahverengi yoksa yeşil)
```

Tuvalde: Üç dallı, gövdesi kahverengi uçları yeşil bir eğrelti otu; dal sayısı sizin oran ve tabanınıza bağlı.

Dizin: `alistirma-koco/kod/KIV-1-K1/` — beklenen çıktı yok, sayı sizin ağacınıza bağlı

IV.2

Koch tanesi

Bir doğru parçası al, ortadaki üçte birini kes, yerine iki kenarlı bir çatı koy; sonra aynı şeyi yeni parçaların her birine yap. Bu sefer küçülen şey derinlik. Her kat çizgi sayısını dörde katlar: derinlik 4'te 768. Dört alıştırma; sayarak çizin.

1 1 1 1

YANIT YERİNE

kitapçık IV.3: Koch tanesi kitapçık kapağı: Koch, Mandelbrot C++ cildi IV: Hanoi'nin sayıları

ISINMA KIV-2-I1 Çizgileri sayın

Kitapçığın kar tanesi, derinlik 4. `ileri`'yi sayan bir sayaç koyun ve yazdırın: 3 × 4⁴ mü? Sonra derinliği 0, 1, 2, 3 yapıp her seferinde sayıyı ve şekli izleyin. Derinlik 10'da sayı kaç olurdu; çizmeye kalkmayın, hesaplayın.

ana.kojo — BAŞLANGIÇ

```
// KIV-2-I1 · Çizgileri sayın
// Çıktı: "derinlik 4: ... çizgi"
tanım koh(boy: Kesir, derinlik: Sayı): Birim =
  eğer (derinlik == 0)
    ileri(boy)
  yoksa {
    koh(boy / 3, derinlik - 1); sol(60)
    koh(boy / 3, derinlik - 1); sağ(120)
    koh(boy / 3, derinlik - 1); sol(60)
    koh(boy / 3, derinlik - 1)
  }
canlandırmaHızınıKur(0)
yinele(3) {
  koh(243, 4)
  sağ(120)
}
```

çıktı — BEKLENEN-ÇİKTİ

```
derinlik 4: 768 çizgi
```

> **Tuvalde:** Kapaktaki kar tanesi: üç koh eğrisi, 768 çizgi.

Dizin: `alistirma-koco/kod/KIV-2-I1/`

ALIŞTIRMA KIV-2-A1

Kare tanesi

Kitapçığın ikinci alıştırması: çatıyı üçgen yerine kare yapın. Parça beşe bölünür: ileri, sol(90) ileri, sağ(90) ileri, sağ(90) ileri, sol(90) ileri; hepsi `boy / 3`. Derinlik 3'te tek bir kenar kaç çizgi? Dört kenarlı kapalı sürümünü de çizin ve sayıyı yazdırın.

çıktı — BEKLENEN-CİKTİ

```
bir kenar, derinlik 3: 125 çizgi
dört kenar: 500 çizgi
```

> **Tuvalde:** Kareli kıvrımlı bir kenar (125 çizgi) ve dört kenarlı kapalı bir kare tanesi (500 çizgi).

Dizin: `alistirma-koco/kod/KIV-2-A1/`

ZORLAYICI KIV-2-Z1

Tanenin çevresi

Her derinlik katı uzunluğu 4/3'e katlar. Derinlik 0..5 için tanenin çevresini iki yoldan hesaplayın: formülle $(3 \times 243 \times (4/3)^d)$ ve çizerek (her `ileri`'nin boyunu bir `den`'e toplayın). İki sütun aynı mı? Çevre sonsuza giderken alan sınırlı kalıyor: neden? (Bir cümle; ipucu: tane çevrel çemberin dışına hiç çıkmaz.)

DÜŞÜNCE

Her derinlik uzunluğu 4/3 katlar; çizilen toplam formülle aynı çıkmalı.

YAKLAŞIM

`(0 |-| 5).herbiriİçin { d => uzunluk = 0.0; yinele(3) { koh(243, d); sağ(120) }; satıryaz(...) }`; formül `3 * 243 * mat.üs(4.0 / 3, d)` ya da bir döngüyle çarpın.

İLK SATIR

```
den formül = 3 * 243.0; yinele(d) { formül = formül * 4 / 3 }
```

çıktı — BEKLENEN-CİKTİ

```
derinlik 0: formül 729 çizilen 729
derinlik 1: formül 972 çizilen 972
derinlik 2: formül 1296 çizilen 1296
derinlik 3: formül 1728 çizilen 1728
derinlik 4: formül 2304 çizilen 2304
derinlik 5: formül 3072 çizilen 3072
```

Tuvalde: Üst üste altı kar tanesi, giderek pürüzlenen; çevreleri 729'dan 3072'ye.

Dizin: `alistirma-koco/kod/KIV-2-Z1/`

KEŞİF KIV-2-K1

Ejderha eğrisi ~2 saat

Kendi fraktalınız. Ejderha eğrisi: `ejderha(boy, derinlik, yön)`; derinlik 0'da düz çizgi; yoksa önce `ejderha(boy, derinlik − 1, 1)`, `yön × 90` derece dön, sonra `ejderha(boy, derinlik − 1, −1)`. (Dönüş `sağ(yön * 90)`; yön 1 ya da −1.) Derinlik 10'da kaç çizgi? Sayın, yazdırın; sonra renkleri derinliğe bağlayın. İnternette "dragon curve" diye aratıp sizinkiyle karşılaştırın.

DÜŞÜNCE

Ejderha: sol kıvrımlı yarı, dönüş, sağ kıvrımlı yarı.

YAKLAŞIM

`ejderha(boy, derinlik - 1, 1); sağ(yön * 90); ejderha(boy, derinlik - 1, -1)`; çizgi sayısı 2^{10}. Renk için `kalemRenginiKur`'u derinliğe bağlayın.

İLK SATIR

```
yoksa { ejderha(boy, derinlik - 1, 1); sağ(yön * 90); ejderha(boy, derinlik - 1, -1) }
```

çıktı — BEKLENEN-CİKTİ

```
derinlik 10: 1024 çizgi
```

Tuvalde: Kendini hiç kesmeyen, kıvrım kıvrım bir ejderha eğrisi; 1024 çizgi.

Dizin: `alistirma-koco/kod/KIV-2-K1/`

IV.3

Hanoi; dizinlerde özyineleme

Üç satırlık Hanoi: $2^n - 1$ hamle. Her dizin ya Boştur ya da baş ve gerisi; `eşle` / `durum` ile tanımlar şiire dönüyor: `toplam`, `altKümeler`. Beş alıştırma; geri alacak bir şey yok, çünkü kimse bir şeyi değiştirmiyor.

1 2 1 1

YANIT YERİNE

kitapçık IV.4: Hanoi, dizinlerde özyineleme algoritma ders 9: Hanoi ders 7 (C++) C++ cildi IV.3-4: altkümeler, permütasyonlar

ISINMA KIV-3-I1 Dört disk

Kitapçığın Hanoi'si üç diskle yedi hamle. `taşı(4, 1, 3, 2)` yapın: kaç hamle, ilk hamle hangi diski nereye götürüyor? Çıktıyı beklenen çıktıyla karşılaştırın. Sonra 1. diskin hamlelerine bakın: hep aynı yönde mi dönüyor? (Dört diskte yön üç diskinkinin tersi; neden?)

ana.kojo — BAŞLANGIÇ

```
// KIV-3-I1 · Dört disk
den adım = 0
tanım taşı(n: Sayı, kaynak: Sayı, hedef: Sayı, yardımcı: Sayı): Birim =
  eğer (n > 0) {
    taşı(n - 1, kaynak, yardımcı, hedef)
    adım = adım + 1
    satıryaz(s"$adım) $n. diski $kaynak -> $hedef")
    taşı(n - 1, yardımcı, hedef, kaynak)
  }
taşı(3, 1, 3, 2)
```

çıktı — BEKLENEN-CİKTİ

```
1) 1. diski 1 -> 2
2) 2. diski 1 -> 3
3) 1. diski 2 -> 3
4) 3. diski 1 -> 2
5) 1. diski 3 -> 1
6) 2. diski 3 -> 2
7) 1. diski 1 -> 2
8) 4. diski 1 -> 3
9) 1. diski 2 -> 3
10) 2. diski 2 -> 1
11) 1. diski 3 -> 1
12) 3. diski 2 -> 3
13) 1. diski 1 -> 2
14) 2. diski 1 -> 3
15) 1. diski 2 -> 3
```

Dizin: `alistirma-koco/kod/KIV-3-I1/`

ALIŞTIRMA KIV-3-A1

Üç tanım, eşle ile

Kitapçığın `toplam`'ı gibi, `eşle` / `durum` ile: `boy(d)` (Boş 0, yoksa 1 + gerisinin boyu), `enBüyük(d)` (tek öge kendisi; yoksa başı ile gerisinin en büyüğünden büyüğü), `ters(d)` (gerisinin tersi, sonuna baş: `:+` ya da `sonunaEkle`). Hazır `boyu`, `enİrisi`, `tersi` ile karşılaştırın.

çıktı — BEKLENEN-CİKTİ

```
boy: 6 6
en büyük: 9 9
ters: 9 5 1 4 1 3 = 9 5 1 4 1 3
```

Dizin: `alistirma-koco/kod/KIV-3-A1/`

ALIŞTIRMA KIV-3-A2

Permütasyonlar

Kitapçığın dördüncü alıştırması: `permütasyonlar(d: Dizin[Sayı]): Dizin[Dizin[Sayı]]`. Taban: boş dizinin tek sıralanışı var, boş. Adım: başı sırayla her öge yapın (`d.işle`), o öge çıkarılmış dizinin (`fark(Dizin(x))`) permütasyonlarının önüne ekleyin, hepsini birleştirin (`düzİşle`). `Dizin(1, 2, 3)` için altısını yazdırın; `belirt` ile 6'yı ve 4 öge için 24'ü sınayın.

çıktı — BEKLENEN-CİKTİ

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
denemeler geçti.
```

Dizin: `alistirma-koco/kod/KIV-3-A2/`

ZORLAYICI İKİLİ KIV-3-Z1

Kümeyi ikiye bölün

Kitapçığın beşinci alıştırması, CSES Elma paylaşımı (*Apple Division*): elmaları iki gruba ayırın, ağırlık farkı en az olsun. `altKümeler` tam bu iş: her altküme bir grup, kalanı öteki; farkların en küçüğü. `Dizin(3, 2, 7, 4, 1)` için yazdırın. Elma sayısı 20'de altküme sayısı kaç; CSES'in 1 saniyesine sığar mı? Toplamlar için `Uzun`.

DÜŞÜNCE

Her altküme bir grup; farkların en küçüğü.

YAKLAŞIM

`dez toplam = elmalar.topla`; `altKümeler(elmalar).işle(ak => (toplam - 2 * ak.topla).mutlak).enUfağı`. 20 elma için 2^{20} altküme, bir milyon; saniyeye sığar.

İLK SATIR

```
dez fark = altKümeler(elmalar).işle(ak => (toplam - 2 * ak.topla).mutlak).enUfağı
```

çıktı — BEKLENEN-CİKTİ

```
en küçük fark: 1
```

Dizin: `alistirma-koco/kod/KIV-3-Z1/`

KEŞİF KIV-3-K1

Hanoi'yi saymak

Hamleleri yazdırmadan sayan `hamle(n)` : taban 0, adım 2 × hamle(n – 1) + 1. 10 ve 20 için `Uzun` yeter; 64 için yetmez ($2^{64} - 1$ > Uzun'un sınırı). `İriSayı` ile yazın ve kitapçığın efsanesini hesaplayın: saniyede bir hamleyle kaç yıl (bir yıl 31 557 600 saniye)? Kitapçık 584 milyar diyor; tam sayı?

DÜŞÜNCE

`hamle(n) = 2 · hamle(n−1) + 1` , taban 0; 64 için İriSayı.

YAKLAŞIM

`tanım hamle(n: Sayı): İriSayı = eğer (n == 0) 0 yoksa 2 * hamle(n - 1) + 1` ; yıl için `hamle(64) / 31557600` .

İLK SATIR

```
tanım hamle(n: Sayı): İriSayı = eğer (n == 0) 0 yoksa 2 * hamle(n - 1) + 1
```

çıktı — BEKLENEN-CİKTİ

```
10: 1023
20: 1048575
64: 18446744073709551615 hamle
584542046090 yıl
```

Dizin: `alistirma-koco/kod/KIV-3-K1/`

IV.4

Unutkan işlevi hızlandırmak

Fibonaççi'yi tanımıyla yazın, `fibYavaş(45)` deyin ve çayınızı koyun. Çare: hesapladığını bir kenara yaz; kenar defteri bir `Eşlem` , `alYoksa(n, {...})` ile. Dört alıştırma; dinamik programlamanın (Bölüm VI) ilk adımı.

1 2 1

YANIT YERİNE

kitapçık IV.5: Unutkan işlevi hızlandırmak Skala kılavuzu: Eşlem dp-sade C++ cildi IV.2: memoizasyon

ISINMA KIV-4-I1 Kaç çağrı?

Kitapçığın iki Fibonaççi'si. İkisine de çağrı sayacı koyun (`den`), `fibYavaş(30)` ve `fibHızlı(30)` için sayıları yazdırın; sonra `fibHızlı(90)` . C++ cildindeki sayılarla aynı mı (2 692 537 ve 59)? Belleğin kaç kaydı var (`bellek.sayı`)?

ana.kojo BAŞLANGIÇ

```
// KIV-4-I1 · Kaç çağrı?
// Çıktı: "yavaş: 832040, ... çağrı", "hızlı: 832040, ... çağrı", "fib(90) = ...",
"bellek: ... kayıt"
tanım fibYavaş(n: Sayı): Uzun =
  eğer (n < 2) n
  yoksa fibYavaş(n - 1) + fibYavaş(n - 2)
dez bellek = Eşlem[Sayı, Uzun]()
tanım fibHızlı(n: Sayı): Uzun =
  eğer (n < 2) n
  yoksa bellek.alYoksa(n, {
    dez sonuç = fibHızlı(n - 1) + fibHızlı(n - 2)
    bellek.eşEkle(n -> sonuç)
    sonuç
  })
satıryaz(fibHızlı(45))
satıryaz(fibHızlı(90))
```

çıktı BEKLENEN-CİKTİ

```
yavaş: 832040, 2692537 çağrı
hızlı: 832040, 59 çağrı
fib(90) = 2880067194370816120
bellek: 89 kayıt
```

Dizin: `alistirma-koco/kod/KIV-4-I1/`

ALIŞTIRMA KIV-4-A1 Collatz, bellekli

KII-2-Z2'nin Collatz adımı, bu sefer özyinelemeli ve bellekli: `adım(n) = 1 + adım(sonraki)` , taban `adım(1) = 0` ; kenar defteri `Eşlem[Uzun, Sayı]` (ara değerler Sayı'yı aşabilir, Uzun kullanın). 10 000'e kadar en uzun zinciri bulun. Bellek olmadan ne kadar sürerdi; deneyin, sabrınız biterse `Durdur` .

çıktı BEKLENEN-CİKTİ

```
en uzun: 6171 (261 adım)
```

Dizin: `alistirma-koco/kod/KIV-4-A1/`

ALIŞTIRMA KIV-4-A2

Pascal üçgeni, bellekli

Binom katsayısı: `binom(n, k) = binom(n−1, k−1) + binom(n−1, k)`, taban k == 0 ya da k == n için 1. Bellek anahtarı bir demet: `Eşlem[(Sayı, Sayı), Uzun]`. `binom(30, 15)`'i yazdırın; sonra belleği kapatıp (alYoksa yerine doğrudan hesap) aynı çağrıyı deneyin: kaç saniye? Bir de üçgenin ilk altı satırını yazdırın.

çıktı — BEKLENEN-CİKTİ

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
binom(30, 15) = 155117520
```

Dizin: `alistirma-koco/kod/KIV-4-A2/`

ZORLAYICI KIV-4-Z1

Uzun'un sınırı, İriSayı

`fibHızlı(90)` Uzun'a ucu ucuna sığıyor; 93 sığmaz. Belleği `Eşlem[Sayı, İriSayı]` yapıp `fibİri(n)` yazın: 93 ve 100'ü yazdırın. Sonra en basit sürümü, iki değişkenli döngüyü (`den a, b`) de yazın ve 100 için aynı sayıyı verdiğini gösterin. Üç sürüm: tanım, tanım + defter, döngü. Hangisi en kısa, hangisi en hızlı, hangisinin kanıtı tanımın kendisi?

DÜŞÜNCE

Aynı fib, türü İriSayı; döngülü sürüm iki değişken.

YAKLAŞIM

`fibİri`: kitapçığın `alYoksa` kalıbı, bellek `Eşlem[Sayı, İriSayı]`. Döngü: `den a: İriSayı = 0; den b: İriSayı = 1`, n kere `(a, b) = (b, a + b)`.

İLK SATIR

```
tanım fibİri(n: Sayı): İriSayı = eğer (n < 2) n yoksa
bellek.alYoksa(n, { dez s = fibİri(n - 1) + fibİri(n - 2);
bellek.eşEkle(n -> s); s })
```

çıktı — BEKLENEN-CİKTİ

```
93: 12200160415121876738
100: 354224848179261915075
döngüyle 100: 354224848179261915075
```

Dizin: `alistirma-koco/kod/KIV-4-Z1/`

Geri dönüş: altkümeler ve permütasyonlar, kümede

Özyineleme üretir, küme toplar. Altküme toplamlarını ve tekrarlı harflerin dizilişlerini bir `Küme`'de biriktirin; tekrarları ve sıralamayı küme halleder.

2

> **YANIT YERİNE**
>
> kitapçık II.3: Küme ve eşlem kitapçık IV.4: Dizinlerde özyineleme

ALIŞTIRMA KIV-5-A1 Altküme toplamları, kümede

`toplamlar(d)`: dizinin bütün altkümelerinin toplamlarını bir `Küme[Sayı]` olarak versin. Taban: boş dizinin tek altkümesi boş, toplamı 0. Adım: kuyruğun toplamlarını al; her birine başı ekleyerek ikinci bir küme kur; ikisini `++` ile birleştir. `Dizin(3, 1, 2, 4)` için kaç farklı toplam var? Kümeyi `dizine.sıralı` ile yazdırın.

çıktı — BEKLENEN-CİKTİ

```
11 farklı toplam
0 1 2 3 4 5 6 7 8 9 10
```

Dizin: `alistirma-koco/kod/KIV-5-A1/`

ALIŞTIRMA KIV-5-A2 Tekrarlı harflerin dizilişleri

`dizilişler(d)`: bir harf dizininin bütün dizilişlerini `Küme[Dizin[Harf]]` olarak versin. Taban: boş dizinin tek dizilişi boş. Adım: `d.yinelemesiz` içindeki her harf için, o harfi bir kere çıkardığınız dizinin (`d.diff(Dizin(h))`) dizilişlerinin önüne harfi ekleyin; kümeleri `indirge` ile birleştirin. `"aab".dizine` için üç diziliş çıkmalı; `yazıYap("")` ile yazıya çevirip sıralı yazdırın.

çıktı — BEKLENEN-CİKTİ

```
3 farklı diziliş
aab
aba
baa
```

Dizin: `alistirma-koco/kod/KIV-5-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`KIV-3-Z1` Kümeyi ikiye bölün

`KIV-1-Z1` Sierpinski üçgeni

`KIV-4-Z1` Uzun'un sınırı, İriSayı

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM V

Çizgeler ve Gezintiler

Kitapçığın beş satırlık gezisi işlevsel kılıkta: değişmez kümeler, soldanKatla, katman katman enlemesine gezi. Dört konu, on sekiz alıştırma; kitapçığın yedi düğümlü çizgesi hemen her soruda. Kümeleri sıralayıp yazdırıyoruz (kıyma kümesi sıra tutmaz); C++ cildinin V. bölümüyle aynı sorular var, iki dilde yan yana koyun.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Soyutlama | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

V.1

Çizge nedir, nasıl saklanır?

Düğümler ve bağlar; neyin neye bağlı olduğu. Her düğümü komşularının kümesine eşleyen bir Eşlem: `tür Çizge = Eşlem[Düğüm, Küme[Düğüm]]`. Çizge de işlev gibi kullanılır: `ç('A')`. Dört alıştırma; dereceler ve Königsberg.

1 2 1

YANIT YERİNE

kitapçık V.1: Çizge nedir çizge kuramı kod: düğüm-çözme.kojo C++ cildi V.1

ISINMA KV-1-I1 Komşuları yazdırın

Kitapçığın çizgesi. Her düğümün komşularını bir satırda yazdırın: düğümler alfabetik (`ç.dizine.sırala(ç => ç._1)`), komşular da sıralı (`dizine.sıralı`). Sonra bir bağ ekleyin, `G -> A` (`eşEkle`), ve yeniden yazdırın. Çizge artık çevrimli: Bölüm V'in tuzaklarından biri bu çizgede bekliyor.

ana.kojo — BAŞLANGIÇ

```
// KV-1-I1 · Komşuları yazdırın
// Çıktı: "A: B C" biçiminde yedi satır, boş satır, yedi satır daha
tür Düğüm = Harf
tür Çizge = Eşlem[Düğüm, Küme[Düğüm]]
dez ç: Çizge = Eşlem(
  'A' -> Küme('B', 'C'),
  'B' -> Küme('D'),
  'C' -> Küme('D', 'E'),
  'D' -> Küme('F'),
  'E' -> Küme('F'),
  'F' -> Küme('G'),
  'G' -> Küme[Düğüm]()
)
// ç.dizine.sırala(...) ile dolaşın
```

çıktı — BEKLENEN-CİKTİ

```
A: B C
B: D
C: D E
D: F
E: F
F: G
G:

A: B C
B: D
C: D E
D: F
E: F
F: G
G: A
```

Dizin: `alistirma-koco/kod/KV-1-I1/`

ALIŞTIRMA KV-1-A1

Dereceler

Yönlü çizgede her düğümün iki derecesi var: çıkan (komşu kümesinin boyu) ve gelen (kaç düğümün kümesinde geçtiği: `ç.say { durum (_, k) => k(d) }`, küme yine işlev). Yedi düğüm için ikisini de yazdırın. Çıkanların toplamı gelenlerin toplamına eşit mi; neden? Kaynak (geleni 0) ve çıkmaz (çıkanı 0) düğümler hangileri?

çıktı — BEKLENEN-CİKTİ

```
A: çıkan 2, gelen 0
B: çıkan 1, gelen 1
C: çıkan 2, gelen 1
D: çıkan 1, gelen 2
E: çıkan 1, gelen 1
F: çıkan 1, gelen 2
G: çıkan 0, gelen 1
toplam: çıkan 8, gelen 8
```

Dizin: `alistirma-koco/kod/KV-1-A1/`

ALIŞTIRMA KV-1-A2

Yönsüze çevirin

Kitapçığın çizgesi yönlü. `yönsüz(ç: Çizge): Çizge` yazın: her A→B bağı için B→A da olsun. Yeni bir `Eşlem` kurun (eskisi değişmesin): her düğüm için komşuları, bir de onu komşu sayan düğümler (`ele` ile). Yönsüz çizgede komşuları yazdırın; derece artık tek: her düğümün derecesini de yazın. Derecelerin toplamı bağ sayısının kaç katı?

çıktı — BEKLENEN-CİKTİ

```
A: B C (2)
B: A D (2)
C: A D E (3)
D: B C F (3)
E: C F (2)
F: D E G (3)
G: F (1)
dereceler toplamı: 16
```

Dizin: `alistirma-koco/kod/KV-1-A2/`

ZORLAYICI KV-1-Z1

Königsberg'in köprüleri

1736: dört kara parçası, yedi köprü; hepsinden tam bir kere geçen yürüyüş var mı? Küme burada yetmez: iki kara parçası arasında iki köprü olabilir. Köprüleri bir `Dizin[(Düğüm, Düğüm)]` olarak tutun. Her düğümün derecesini (kaç köprüde geçiyor) `say` ile hesaplayın; tek dereceli düğüm sayısını yazdırın. Euler: 0 ya da 2 ise yol var. Köprüler: A-B, A-B, A-C, A-C, A-D, B-D, C-D.

DÜŞÜNCE

Derece: düğümün kaç köprüde geçtiği; iki uçta da geçebilir.

YAKLAŞIM

`köprüler.say { durum (a, b) => a == d || b == d }`; dört düğüm için yazdırın; tek dereceli sayısı `Dizin('A','B','C','D').say(d => derece(d) % 2 == 1)`.

İLK SATIR

```
tanım derece(d: Düğüm): Sayı = köprüler.say { durum (a, b) => a == d || b == d }
```

çıktı — BEKLENEN-CİKTİ

```
A: 5
B: 3
C: 3
D: 3
tek dereceli: 4, Euler yolu yok
```

Dizin: `alistirma-koco/kod/KV-1-Z1/`

V.2

Derinlemesine gezi, iki kılıkta

`soldanKatla`: başlangıç değeri + birleştirme işlevi. İşlevsel gezi dört satır: uğramışsak elimizdeki küme, yoksa kendimizi ekleyip her komşunun getirdiklerini üst üste katla. Aynı gezi yığınla; yığın kuyruk olunca bambaşka bir algoritma. Beş alıştırma.

1 2 1 1

YANIT YERİNE

kitapçık V.2: Derinlemesine gezi algoritma ders 16: derinlemesine gezi C++ cildi V.2: gezi kalıbı

ISINMA KV-2-I1 Üç yerden gezi

Kitapçığın işlevsel `gez`'i. A'dan başlayınca yedi düğümün yedisi; D'den, G'den başlayınca kaçı? Üç geziyi de kümeyi sıralayıp yazdırın. `gez` hiç `den` kullanmıyor; gezilenler nerede birikiyor? (Her çağrının döndürdüğü kümeyi izleyin.)

ana.kojo — BAŞLANGIÇ

```
// KV-2-I1 · Üç yerden gezi
// Çıktı: "A: A B C D E F G", "D: ...", "G: ..."
tür Düğüm = Harf
tür Çizge = Eşlem[Düğüm, Küme[Düğüm]]
dez ç: Çizge = Eşlem(
  'A' -> Küme('B', 'C'),
  'B' -> Küme('D'),
  'C' -> Küme('D', 'E'),
  'D' -> Küme('F'),
  'E' -> Küme('F'),
  'F' -> Küme('G'),
  'G' -> Küme[Düğüm]()
)
tanım gez(ç: Çizge, buradan: Düğüm, gezilen: Küme[Düğüm]): Küme[Düğüm] =
  eğer (gezilen(buradan)) gezilen
  yoksa ç(buradan).soldanKatla(gezilen + buradan)(
    (g, komşu) => gez(ç, komşu, g)
  )
satıryaz(s"A: ${gez(ç, 'A', Küme[Düğüm]()).dizine.sıralı.yazıYap(" ")}")
```

çıktı — BEKLENEN-CİKTİ

```
A: A B C D E F G
D: D F G
G: G
```

Dizin: `alistirma-koco/kod/KV-2-I1/`

ALIŞTIRMA KV-2-A1 Yol var mı?

Kitapçığın ikinci alıştırması: `ulaşırMı(ç, a, b): İkil`. `gez` zaten her şeyi biliyor; tek satır: a'dan gezilenler kümesi b'yi içeriyor mu (küme işlev!). A→G, G→A, B→E, E→F için evet/hayır. Bir düğüm kendine ulaşır mı; bu tanıma göre ne çıkıyor, doğru mu?

çıktı — BEKLENEN-CİKTİ

```
A -> G: evet
G -> A: hayır
B -> E: hayır
E -> F: evet
```

Dizin: `alistirma-koco/kod/KV-2-A1/`

ALIŞTIRMA KV-2-A2 Yığın mı, kuyruk mu?

Kitapçığın birinci alıştırması: `gez2`'yi kuyrukla yazın (`koy`→`ekle`, `al`→`baştanAl`) ve iki çıktıyı alt alta koyun. Kitapçığın çıktısıyla aynı mı: `A C E F G D B` ve `A B C D E F G`? Hangi sıra hangi geziye ait, bakar bakmaz söyleyin: biri derinlere dalıyor, öteki katman katman.

çıktı — BEKLENEN-CİKTİ

```
yığınla : A C E F G D B
kuyrukla: A B C D E F G
```

Dizin: `alistirma-koco/kod/KV-2-A2/`

ZORLAYICI İKİLİ KV-2-Z1 Gerçek bir hata, sayılarla

Kitapçığın "Gerçek bir hata"sı: `eğer (!gezilen(şimdiki))` silinirse iki yoldan ulaşılan düğümler iki kere işlenir, çevrimde ise hiç durmaz. Ölçün: `gez2`'nin denetimsiz bir sürümünü yazın ama en çok `sınır` işlemde dursun ve kaç işlem yaptığını döndürsün. Kitapçığın çizgesinde (çevrimsiz) kaç işlem? Denetimli 7; sonra `G -> A` bağını ekleyin: sınıra (1000) çarpıyor mu?

DÜŞÜNCE

Denetimsiz gezi aynı düğümü yeniden işler; çevrimde hiç durmaz, sınırla durdurun.

YAKLAŞIM

Yığınlı `gez2`'yi kopyalayın; `eğer (!gezilen(şimdiki))` satırını kaldırın, döngüye `işlem < sınır` koşulu ekleyin, işlem sayısını döndürün. `G -> A` bağı çevrimi yaratır.

İLK SATIR

`yineleDoğruysa(yığın.boyu > 0 && işlem < sınır) { ... işlem = işlem + 1 }`

çıktı — BEKLENEN-CİKTİ

```
denetimsiz: 12 işlem
denetimli: 7 işlem
çevrimle: 1000 işlemde durduruldu
```

Dizin: `alistirma-koco/kod/KV-2-Z1/`

KEŞİF KV-2-K1

Çizgeyi tuvale çizin

Kitapçığın “Takım işi”: `düğümÇiz(ad, x, y)` (`atla` , küçük `daire` , yanına `yazı(ad)`) ve iki düğüm arasına çizgi çeken `bağÇiz` (`atla` , `noktayaDön` , uzaklık kadar `ileri` ; uzaklık `karekökü` ile). Konumları bir `Eşlem[Düğüm, (Kesir, Kesir)]` 'de tutun; yedi düğümlü çizgeyi ekrana alın. Kaç düğüm, kaç bağ çizdiniz, yazdırın. Gezilerinizi resmin üstünde parmakla izleyin.

DÜŞÜNCE

Düğümlerin yerini siz seçersiniz; bağ iki konum arası düz çizgi.

YAKLAŞIM

`konum: Eşlem[Düğüm, (Kesir, Kesir)]` ; `düğümÇiz` : `atla(x, y); daire(10); yazı(ad)` ; `bağÇiz` : `atla(x1, y1); noktayaDön(x2, y2); ileri(karekökü(...))` . Her bağı bir kere çizmek için `a < b` koşulu.

İLK SATIR

```
tanım bağÇiz(a: Düğüm, b: Düğüm) = { dez (x1, y1) = konum(a); dez (x2, y2) = konum(b); atla(x1, y1); noktayaDön(x2, y2); ileri(karekökü((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1))) }
```

çıktı — BEKLENEN-CİKTİ

```
7 düğüm, 8 bağ
```

Tuvalde: Soldan sağa akan yedi düğüm (A'dan G'ye), aralarında sekiz çizgi; her düğüm küçük bir daire ve harfi.

Dizin: `alistirma-koco/kod/KV-2-K1/`

V.3

Katman katman: en az adım; kaç parça?

0. katman başlangıç, 1. katman komşuları, 2. katman onların görülmemiş komşuları... `düzİşle` ile. Bir düğüm hangi katmanda ilk görünüyorsa oraya daha kısa yol olamaz; kanıt kodun şeklinde. Sonra parça saymak: `soldanKatla` 'nın taşıdığı bir demet. Beş alıştırma.

`1` `2` `1` `1`

YANIT YERİNE

kitapçık V.3: Katman katman algoritma ders 15: enlemesine gezi algoritma ders 17: bağlı parçalar C++ cildi V.3-4

ISINMA KV-3-I1 Katmanları yazdırın

Kitapçığın `katmanlar`'ı, kümeler sıralı yazdırılmış hâliyle. A'dan beş katman; sonra C'den başlayın: kaç katman, G hangisinde? Üçüncü girdi `gezilen` neden başlangıçta `Küme(başlangıç)`; boş küme verince ne olur (deneyin, açıklayın)?

ana.kojo — BAŞLANGIÇ

```
// KV-3-I1 · Katmanları yazdırın
// Çıktı: "0 adım: A" biçiminde katmanlar; A'dan, sonra C'den
tür Düğüm = Harf
tür Çizge = Eşlem[Düğüm, Küme[Düğüm]]
dez ç: Çizge = Eşlem(
  'A' -> Küme('B', 'C'),
  'B' -> Küme('D'),
  'C' -> Küme('D', 'E'),
  'D' -> Küme('F'),
  'E' -> Küme('F'),
  'F' -> Küme('G'),
  'G' -> Küme[Düğüm]()
)
tanım katmanlar(ç: Çizge, sınır: Küme[Düğüm], gezilen: Küme[Düğüm]): Dizin[Küme[Düğüm]]
=
  eğer (sınır.boşMu) Boş
  yoksa {
    dez sonraki = sınır.düzİşle(d => ç(d)).ele(k => !gezilen(k))
    sınır :: katmanlar(ç, sonraki, gezilen ++ sonraki)
  }
dez kat = katmanlar(ç, Küme('A'), Küme('A'))
kat.ikileSırayla.herbiriİçin { durum (katman, uzaklık) =>
  satırYaz(s"$uzaklık adım: ${katman.dizine.sıralı.yazıYap(" ")}")
}
```

çıktı — BEKLENEN-CİKTİ

```
başlangıç A:
0 adım: A
1 adım: B C
2 adım: D E
3 adım: F
4 adım: G
başlangıç C:
0 adım: C
1 adım: D E
2 adım: F
3 adım: G
```

Dizin: `alistirma-koco/kod/KV-3-I1/`

ALIŞTIRMA KV-3-A1

En az kaç adım?

`uzaklık(ç, a, b): Sayı`: a'dan katmanları alın, b'nin ilk geçtiği katmanın sırası (`nerede(k => k(b))`, bulamazsa −1 verir, tam istediğimiz). A→G, A→E, D→G, G→A için yazdırın. Neden bu en az adım; kitapçığın tek cümlelik kanıtını kendi sözlerinizle yazın.

çıktı — BEKLENEN-CİKTİ

```
A -> G: 4
A -> E: 2
D -> G: 2
G -> A: -1
```

Dizin: `alistirma-koco/kod/KV-3-A1/`

ALIŞTIRMA KV-3-A2

Kaç parça?

Yönsüz bir çizge: A-B, A-C, B-D, C-D ve ayrı bir E-F. Kitapçığın parça sayan `soldanKatla`'sını (demet taşıyan) kurun; 2 parça çıkmalı. Sonra D-E bağını (iki yönlü!) ekleyin: 1 parça. Yönsüz çizgeyi iki yönlü girerek kurduğunuza dikkat.

çıktı — BEKLENEN-CİKTİ

```
2 parça
1 parça
```

Dizin: `alistirma-koco/kod/KV-3-A2/`

ZORLAYICI KV-3-Z1

Neden sıraladık?

Kitapçığın “Neden sıraladık?” kutusu: yönlü çizgede parça sayısı düğümleri hangi sırayla denediğinize bağlı. Kitapçığın yönlü çizgesinde parça sayısını iki kere hesaplayın: düğümler alfabetik ve tersten (`sıralı.tersi`). İki sayıyı yazdırın. Fark neden? Bir cümleyle: “bağlı parça” hangi çizgelerin kavramı?

DÜŞÜNCE

Yönlü çizgede “parça” başlangıca bağlı; sıra yanıtı değiştirir.

YAKLAŞIM

Parça sayan işlevi düğüm dizinini girdi alacak biçimde yazın; `ç.anahtarlar.dizine.sıralı` ve `.tersi` ile iki kere çağırın. (Anahtar dizini için `ç.dizine.işle(_._1)` de olur.)

İLK SATIR

```
tanım parçaSay(sıra: Dizin[Düğüm]): Sayı = ...
```

çıktı — BEKLENEN-CİKTİ

```
alfabetik: 1 parça
tersten: 7 parça
```

Dizin: `alistirma-koco/kod/KV-3-Z1/`

KEŞİF KV-3-K1

Labirentte katmanlar

Çizge yazılı olmak zorunda değil: bir labirentin kareleri düğüm, yan yana boş kareler bağ. `katmanlar`'ı genelleştirin: çizge yerine bir `komşular: Kare => Küme[Kare]` işlevi alsın (`tür Kare = (Sayı, Sayı)`). CSES'in Labyrinth örneğinde A'dan B'ye en az kaç adım? (C++ cildinde aynı soru kuyrukla: `V-4-A1`, yanıt 9.) Plan: satırlar bir `Dizin[Yazı]`; `#` duvar.

DÜŞÜNCE

Çizge bir işlev: kareden komşu kümesine; gerisi kitapçığın katmanlarıyla aynı.

YAKLAŞIM

`katmanlar(komşular, Küme(a), Küme(a), 0)`: her katmanda sınırın komşularını alıp gezilenleri çıkarın; `B` sınıra girince adımı döndürün. `A` ve `B`'nin karesini planda arayın.

İLK SATIR

```
dez yeniSınır = sınır.düzİşle(komşu).ele(k => !gezilen(k))
```

çıktı — BEKLENEN-CİKTİ

```
A'dan B'ye 9 adım
```

Dizin: `alistirma-koco/kod/KV-3-K1/`

V.4

Yollar eşit değilse

Bağların ağırlığı olunca kuyruğun yerini öncelik sırası alır, algoritmanın adı Dijkstra olur. Kitapçık bu kodu bilerek yazmıyor; kardeş kitapçıktaki sürümü Koco'ya çevirmek sizin işiniz. KII-4-Z1'in eksi numarasını hatırlayın. Dört alıştırma.

1 1 1 1

YANIT YERİNE

kitapçık V.4: Yollar eşit değilse algoritma ders 23: Dijkstra temel-bilgiler 7: oyun tarifleri C++ cildi V.7: Dijkstra

ISINMA KV-4-I1

Ağırlıklı çizge

Yollar: her şehirden (komşu, süre) ikililerinin dizini: `tür Yollar = Eşlem[Sayı, Dizin[(Sayı, Sayı)]]`. Kardeş kitapçığın üç şehirlik örneği: 1→2 (6), 1→3 (2), 3→2 (3). Kurun; her şehrin yollarını yazdırın. Sonra kâğıtta: 1'den 2'ye en kısa süre kaç? Doğrudan 6, dolaylı 2 + 3 = 5; programa hesaplatmadan önce elle bulun ve yazdırın.

ana.kojo — BAŞLANGIÇ

```
// KV-4-I1 · Ağırlıklı çizge
// Çıktı: "1: 2(6) 3(2)", "2: -", "3: 2(3)", "1'den 2'ye elle: 5"
tür Yollar = Eşlem[Sayı, Dizin[(Sayı, Sayı)]]
dez yollar: Yollar = Eşlem(1 -> Dizin((2, 6), (3, 2)), 2 -> Boş, 3 -> Dizin((2, 3)))
// yazdırın
```

çıktı — BEKLENEN-CİKTİ

```
1: 2(6) 3(2)
2: -
3: 2(3)
1'den 2'ye elle: 5
```

Dizin: `alistirma-koco/kod/KV-4-I1/`

ALIŞTIRMA KV-4-A1

Dijkstra'yı Koco'ya çevirin

Kitapçığın beşinci alıştırması. İskelet gezilerle aynı: bekleyenlerden birini al, işle, komşularını beklet; tek fark “hangisini önce”: en küçük süreyi. `ÖncelikSırası[(Sayı, Sayı)]` en büyüğü verdiği için (–süre, şehir) koyun. Uzaklıkları bir `Eşlem[Sayı, Sayı]`'de tutun; eski kayıt gelirse atlayın. Üç şehir için 0 5 2 çıkmalı.

çıktı — BEKLENEN-CİKTİ

```
1: 0
2: 5
3: 2
```

Dizin: `alistirma-koco/kod/KV-4-A1/`

ZORLAYICI İKİLİ KV-4-Z1

Yolu da yazın

Uzaklık yetmez, yol da lazım. Dijkstra'ya ikinci bir eşlem ekleyin: `nereden`, her şehre en kısa yolda hangi şehirden gelindiği. Sonunda hedeften geriye izleyip (başlangıca gelene kadar) dizini ters çevirin. Kardeş kitapçığın beş şehirlik örneği: 1→2 (4), 1→3 (1), 3→2 (2), 2→4 (5), 3→4 (8), 4→5 (3), 3→5 (10). Uzaklıklar 0 3 1 8 11; 1'den 5'e yol? Dikkat: 11'e iki yol var (1 3 5 ve 1 3 2 4 5); sizinki hangisini buluyor, neden? (Kardeş kitapçık ötekini buldu.)

DÜŞÜNCE

Uzaklığı iyileştiren her gevşetme, nereden geldiğini de yazsın.

YAKLAŞIM

Dijkstra'da `eğer (yeni < uzaklık.alYoksa(şehir, SONSUZ))` dalında `nereden.eşEkle(şehir -> buradan)`; sonunda 5'ten 1'e `nereden` ile geri yürüyüp `tersi`.

İLK SATIR

```
den p = 5; den yol: Dizin[Sayı] = Dizin(5); yineleDoğruysa(p != 1) { p = nereden(p); yol = p :: yol }
```

çıktı — BEKLENEN-ÇİKTİ

```
uzaklıklar: 0 3 1 8 11
1'den 5'e: 1 3 5
```

Dizin: `alistirma-koco/kod/KV-4-Z1/`

KEŞİF KV-4-K1 Düğüm çözme oyununa ağırlık ~3 saat

Kitapçığın düğüm çözme oyunu (`kitapcik-koco/kod/düğüm-çözme.kojo`; iKojo'da) düğümleri sürükleyip bağların kesişmesini önletiyor. Oyuna bir ölçü ekleyin: bütün bağların toplam uzunluğunu (ya da en uzununu) yazdıran bir düğme; her sürüklemeden sonra güncellensin. Kazanan: kesişmeyen ve en kısa toplam. Sonra tartışın: "en kısa toplam" ile "en kısa yol" aynı şey mi? Bu dizin sizin; betiği kopyalayıp başlayın.

DÜŞÜNCE

Toplam uzunluk, her bağın iki ucunun uzaklıklarının toplamı; her çizimden sonra yeniden hesap.

YAKLAŞIM

`bağlarıÇiz` zaten uzunluğu hesaplıyor; bir `den toplam`'a ekleyin ve sonunda `yazı` ile tuvale yazın. Düğme yerine her sürüklemede yeniden çizmek yeter.

İLK SATIR

`den toplam = 0.0` ve `bağlarıÇiz`'in sonunda `yazı(s"toplam: ${toplam.yuvarla}")`

Dizin: `alistirma-koco/kod/KV-4-K1/` — beklenen çıktı yok

V.5

Geri dönüş: eşlemle çizge, özyinelemeyle oda

Çizge kodunun altında eski tanıdıklar var: komşuluk bir eşlem, derinlemesine gezi bir özyineleme. İki kısa soru, ikisi de o tanıdıkları adıyla çağırıyor.

2

YANIT YERİNE

kitapçık II.3: Küme ve eşlem kitapçık IV.1: Taban ve adım kitapçık V.1: Çizge nedir kitapçık V.3: Kaç parça

ALIŞTIRMA KV-5-A1

Dereceler, eşlemle

Bağlar bir çift dizini: `Dizin(("A","B"), ("A","C"), ("B","C"), ("C","D"), ("D","E"))`. Boş bir `Eşlem[Yazı, Küme[Yazı]]` açıp her bağı iki yöne de ekleyin (`alYoksa` ile eski kümeyi alın, `+` ile büyütün, `eşEkle`). Çıktı: her düğüm ve derecesi ada göre sıralı, son satırda en çok komşusu olan düğüm. Küme sayesinde aynı bağ iki kere gelse de derece şişmez.

çıktı — BEKLENEN-CİKTİ

```
A: 2
B: 2
C: 3
D: 2
E: 1
en çok komşu: C (3)
```

Dizin: `alistirma-koco/kod/KV-5-A1/`

ALIŞTIRMA KV-5-A2

Odaları özyinelemeyle sayın

Harita bir `Dizin[Yazı]`: `.` boş, `#` duvar. `doldur(s, k)` IV. bölümün özyinelemesiyle bir odayı dolaşsın: taban sınır dışı, duvar ya da zaten gezilmiş; değilse kareyi gezilen kümesine ekleyip dört komşuyu çağırsın ve kare sayısını döndürsün. Gezilen küme değişmez olsun, `den` ile yeniden bağlayın. Çıktı: oda sayısı ve en büyük odanın kare sayısı.

çıktı — BEKLENEN-CİKTİ

```
4 oda
en büyük oda: 4 kare
```

Dizin: `alistirma-koco/kod/KV-5-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`KV-1-Z1` Königsberg'in köprüleri

`KV-3-K1` Labirentte katmanlar

`KV-4-Z1` Yolu da yazın

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM VI

Dinamik Programlama

Tümevarım formülü zaten bir işlev tanımı: kenar defteriyle ızgara yolları, bozuk paralar; sonra kendi türünüz ve Mandelbrot; sonunda yola devam. Dört konu, on dokuz alıştırma. Mandelbrot'u bir kere de metinle çiziyoruz: beklenen çıktı bir resim.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Soyutlama · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

VI.1

Izgarada kaç yol; işlevsel çizelge

Bir kareye ya üstten ya soldan gelinir: `yol(r, c) = yol(r−1, c) + yol(r, c−1)`. Kenar defteriyle her kare bir kere. Sonra aşağıdan yukarı: her satır bir öncekinin birikimli toplamı, `soldanKatla` ile. Beş alıştırma; zar dizileri de burada.

1 2 1 1

> YANIT YERİNE
>
> kitapçık VI.1-2: Izgarada kaç yol dp-sade algoritma ders 27 C++ cildi VI.2

ISINMA KVI-1-I1 Defterde kaç kayıt?

Kitapçığın `yol`'u. 20×20'yi yazdırın; sonra `bellek.sayı`: kitapçık 400 diyor; 21×21 = 441 kare varken neden? (Taban durumları deftere yazılıyor mu?) Belleği kapatıp (`alYoksa` yerine doğrudan toplam) `yol(12, 12)`'yi deneyin: ne kadar sürüyor? 20×20'yi denemeyin.

ana.kojo — BAŞLANGIÇ

```
// KVI-1-I1 · Defterde kaç kayıt?
// Çıktı: "20x20: 137846528820" ve "kayıt: ..."
dez bellek = Eşlem[(Sayı, Sayı), Uzun]()
tanım yol(r: Sayı, c: Sayı): Uzun =
  eğer (r == 0 || c == 0) 1
  yoksa bellek.alYoksa((r, c), {
    dez s = yol(r - 1, c) + yol(r, c - 1)
    bellek.eşEkle((r, c) -> s)
    s
  })
satıryaz(s"20x20: ${yol(20, 20)}")
```

çıktı — BEKLENEN-CİKTİ

```
20x20: 137846528820
kayıt: 400
```

Dizin: `alistirma-koco/kod/KVI-1-I1/`

ALIŞTIRMA KVI-1-A1 Birikimli toplam, dikdörtgen

Kitapçığın `birikimli`'si ve `yinele(20) { satır = birikimli(satır) }`. Bunu bir işleve koyun: `yollarAY(r: Sayı, c: Sayı): Uzun` (c + 1 birlik satır, r kere birikimli, sonu). 2×2, 3×4 ve 20×20 için yazdırın; sonuncusu `yol(20, 20)`'yle aynı olmalı. Hangisi daha az bellek kullanıyor, neden?

çıktı — BEKLENEN-CİKTİ

```
2x2: 6
3x4: 35
20x20: 137846528820
```

Dizin: `alistirma-koco/kod/KVI-1-A1/`

ALIŞTIRMA KVI-1-A2

Izgarada engel

Kitapçığın üçüncü son alıştırması: yasak kareler. Yasakları bir `Küme[(Sayı, Sayı)]`'de tutun; tümevarımın tek satırı değişiyor: yasak karede `yol = 0`. Bu sefer sol üstten sayın: `yol(0, 0) = 1`, ızgara dışı 0. AtCoder DP H'nin örneği: 3 satır, 4 sütun, yasaklar (0, 3) ve (1, 1); sağ alta kaç yol? (C++ cildinde aynı örnek: 3.)

çıktı — BEKLENEN-CİKTİ

```
3x4, iki engel: 3 yol
```

Dizin: `alistirma-koco/kod/KVI-1-A2/`

ZORLAYICI KVI-1-Z1

Zar dizileri

Kitapçığın birinci son alıştırması, CSES DP 1: toplamı n olan kaç zar dizisi? f(0) = 1, f(n) = f(n−1) + … + f(n−6), eksi n için 0. Bu bölümün kalıbıyla: `(1 |-| 6).işle(z => f(n − z)).topla` ve kenar defteri. f(0)..f(10)'u tek satırda yazdırın; sonra f(100)'ü `İriSayı` ile (kaç basamak?).

DÜŞÜNCE

Altı önceki değerin toplamı; eksi için 0, sıfır için 1; defter İriSayı.

YAKLAŞIM

`eğer (n < 0) 0 yoksa eğer (n == 0) 1 yoksa bellek.alYoksa(n, { dez s = (1 |-| 6).işle(z => f(n - z)).topla; bellek.eşEkle(n -> s); s })`; basamak: `f(100).yazıya.boyu`.

İLK SATIR

```
dez s = (1 |-| 6).işle(z => f(n - z)).topla
```

çıktı — BEKLENEN-CİKTİ

```
1 1 2 4 8 16 32 63 125 248 492
f(100) = 29007847991461058782363004409 8 (30 basamak)
```

Dizin: `alistirma-koco/kod/KVI-1-Z1/`

KEŞİF KVI-1-K1

Zar kombinasyonları, bir milyon

CSES 1633 (*Dice Combinations*): n bir milyona kadar, yanıt mod $10^9 + 7$. Özyineleme bir milyon derinliğe inemez (yığın taşar); aşağıdan yukarı gidin: son altı değeri bir `Dizin[Uzun]`'da taşıyan bir `soldanKatla` ile (`(1 |-| n)` üstünde). Her adımda yeni değer = altının toplamı mod, dizin kaydırılır (`al(6)`). n = 10^6 için yazdırın; C++ cildiyle aynı sayı çıkmalı (874273980). CSES'e gönderin.

DÜŞÜNCE

Bir milyon adım döngü; her adımda yalnız son altı değer gerekir.

YAKLAŞIM

`(1 |-| n).soldanKatla(Dizin(1L, 0L, 0L, 0L, 0L, 0L)) { (son, _) => (son.topla % MOD) :: son.al(5) }`; sonunda başı.

İLK SATIR

```
(1 |-| n).soldanKatla(Dizin(1L, 0L, 0L, 0L, 0L, 0L)) { (son, _) => (son.topla % MOD) :: son.al(5) }.başı
```

çıktı — BEKLENEN-CİKTİ

```
f(1000000) mod 1e9+7 = 874273980
```

Dizin: `alistirma-koco/kod/KVI-1-K1/`

VI.2

En az kaç bozuk para?

Aceleci 7 + 1 + 1 + 1 + 1 der, doğrusu 5 + 5 + 1. Son parayı sorun: `1 + paralar.işle(p => enAzPara(tutar − p)).enUfağı`, formülün birebir çevirisi. “Sonsuz” için kocaman sayı. Altı alıştırma; paraların dökümü, diziler ve kümeler, Belki ile dürüst sonsuz.

1 2 2 1

YANIT YERİNE

kitapçık VI.3: En az kaç bozuk para algoritma ders 28: DP kanıtları algoritma ders 29: kanıttan koda C++ cildi VI.4

ISINMA KVI-2-I1 Aceleci yanılsın

Kitapçığın `enAzPara`'sı: 11 için 3. Programı 8 için de çalıştırın; sonra paraları `Dizin(1, 4, 6)` yapıp 8'i yeniden: 2 çıkmalı (4 + 4), aceleci 3 derdi (6 + 1 + 1). Dikkat: paralar değişince defteri de boşaltmak gerekir (`defter.boşalt()`) — neden?

ana.kojo — BAŞLANGIÇ

```
// KVI-2-I1 · Aceleci yanılsın
// Çıktı: "1,5,7 ile 11: 3", "1,5,7 ile 8: ...", "1,4,6 ile 8: 2"
dez paralar = Dizin(1, 5, 7)
dez defter = Eşlem[Sayı, Sayı]()
tanım enAzPara(tutar: Sayı): Sayı =
  eğer (tutar == 0) 0
  yoksa eğer (tutar < 0) 1000000
  yoksa defter.alYoksa(tutar, {
    dez s = 1 + paralar.işle(p => enAzPara(tutar - p)).enUfağı
    defter.eşEkle(tutar -> s)
    s
  })
satıryaz(s"1,5,7 ile 11: ${enAzPara(11)}")
```

çıktı — BEKLENEN-CİKTİ

```
1,5,7 ile 11: 3
1,5,7 ile 8: 2
1,4,6 ile 8: 2
```

Dizin: `alistirma-koco/kod/KVI-2-I1/`

ALIŞTIRMA KVI-2-A1 Hangi paralar?

Kitapçığın ikinci son alıştırması: sayı yerine döküm. `enAzParalar(tutar): Dizin[Sayı]`: tutar 0 için Boş; eksi için "imkânsız" anlamına çok uzun bir dizin (1000 tane 1 gibi); yoksa her p için `p :: enAzParalar(tutar - p)`, en kısası (`enUfağı(d => d.boyu)`). 11 ve 23 için dökümü ve boyunu yazdırın. Birden çok en kısa döküm olabilir; sizinki hangisini seçiyor?

çıktı — BEKLENEN-CİKTİ

```
11: 1 5 5 (3 para)
23: 1 1 7 7 7 (5 para)
```

Dizin: `alistirma-koco/kod/KVI-2-A1/`

ALIŞTIRMA KVI-2-A2

Diziler mi, kümeler mi?

Kardeş kitapçığın üçüncü sorusu, işlevsel kılıkta. Paralar 2, 3, 5; hedef 9. Dizi sayısı: f(x) = Σ f(x − p), f(0) = 1, eksi 0 (sıra önemli: 8). Küme sayısı: paraları sırayla kullanın, g(x, k) = "yalnız ilk k parayla x": g(x, k) = g(x, k−1) + g(x − p_k, k) (sıra önemsiz: 3). İkisini de kenar defteriyle yazın, 9 için yazdırın. Döngü sırası yerine özyineleme sırası.

çıktı — BEKLENEN-ÇİKTİ

```
dizi: 8
küme: 3
```

Dizin: `alistirma-koco/kod/KVI-2-A2/`

ZORLAYICI İKİLİ KVI-2-Z1

Aceleci ne zaman doğru?

Gerçek paralarla (1, 5, 10, 25) aceleci yöntem hep doğru; (1, 5, 7) ile değil. Sınatın: `aceleci(tutar, paralar)` büyükten küçüğe sığdıkça alsın; `enAzPara`'yı paraları girdi alan hâle getirin (defter de her para kümesi için ayrı). 1..100'de kaç tutarda ayrışıyorlar, ilk ayrışan tutar hangisi? İki para kümesi için yazdırın. (C++ cildinde aynı deney: 26 fark, ilki 10.)

DÜŞÜNCE

Aceleci: büyükten küçüğe sığdıkça; DP: kitapçığın enAzPara'sı, paraları girdi alan.

YAKLAŞIM

`aceleci`: `paralar.sıralı.tersi.soldanKatla((tutar, 0))` ya da döngü. `enAz`: defteri işlevin içinde açın ki her para kümesi için yeni olsun. `(1 |-| 100).ele(t => aceleci(t, p) != enAz(t, p))`.

İLK SATIR

```
dez farklar = (1 |-| 100).ele(t => aceleci(t, paralar) != enAz(t, paralar))
```

çıktı — BEKLENEN-ÇİKTİ

```
1 5 10 25: 0 fark
1 5 7: 26 fark, ilki 10
```

Dizin: `alistirma-koco/kod/KVI-2-Z1/`

ZORLAYICI KVI-2-Z2

Sonsuz yerine Belki

“Kocaman sayı” bir hile. Dürüst sürüm: `enAz(tutar): Belki[Sayı]`; eksi tutar `Hiçbiri`, 0 `Biri(0)`, yoksa komşu sonuçlardan dolu olanların en küçüğünün bir fazlası, hiç dolu yoksa `Hiçbiri`. Paralar 5 ve 7: 11 için “yok”, 12 için 2, 23 için ? (`ele(_.varMı)`, `işle(_.al)`; ya da `düzleştir`.)

DÜŞÜNCE

“Yok” bir değer: `Hiçbiri`; en küçüğü dolu olanlar arasından seçin.

YAKLAŞIM

`paralar.işle(p => enAz(tutar - p)).düzleştir` dolu sonuçların dizini; boşsa `Hiçbiri`, değilse `Biri(1 + dizin.enUfağı)`. Defter `Belki` saklar.

İLK SATIR

```
dez adaylar = paralar.işle(p => enAz(tutar - p)).düzleştir
```

çıktı — BEKLENEN-CİKTİ

```
11: yok
12: 2
23: yok
```

Dizin: `alistirma-koco/kod/KVI-2-Z2/`

KEŞİF KVI-2-K1 En az para, aşağıdan yukarı

CSES 1634 (*Minimizing Coins*): tutar bir milyona kadar; özyineleme derinliği yetmez. Aşağıdan yukarı: `Dizik.doldur(x + 1)(SONSUZ)`, `f(0) = 0`, 1'den x'e her t için paraları gezip `enUfağı`. İmkânsızsa −1. CSES'in örneği (1, 5, 7 ile 11) → 3; sonra 1 000 000 için (1, 5, 7). Gönderin; dizik ile Eşlem arasındaki hız farkını da ölçün (`buSaniye`).

DÜŞÜNCE

Çizelge bir dizik; her tutar için her parayı dene, en küçüğünü yaz.

YAKLAŞIM

`(1 |-| x).herbiriİçin { t => paralar.herbiriİçin { p => eğer (t - p >= 0 && f(t - p) + 1 < f(t)) f(t) = f(t - p) + 1 } }`.

İLK SATIR

```
eğer (t - p >= 0 && f(t - p) + 1 < f(t)) f(t) = f(t - p) + 1
```

çıktı — BEKLENEN-CİKTİ

```
11: 3
1000000: 142858
```

Dizin: `alistirma-koco/kod/KVI-2-K1/`

VI.3

Kendi türünüz: karmaşık sayılar

`durum sınıf Karmaşık(g, s)`, iki satırlık `+` ve `*`; kendi türünüz sayılar kadar rahat. Mandelbrot'un kuralı tek satır: z ← z² + c; yüz adımda kaçmazsa içeride. Dört alıştırma; biri resmi metinle çiziyor, biri tuvale boyuyor.

1 1 1 1

YANIT YERİNE

kitapçık VI.4: Kendi türünüz Skala kılavuzu: durum sınıf temel-bilgiler 5: renkler C++ cildi II.5: kendi türünüz

ISINMA KVI-3-I1

Bir işleç daha

Kitapçığın `Karmaşık`'ı: `a + a` ve `a * a`. Bir de `-` ekleyin (çıkarma) ve `büyüklüğü`'nü yazın. Sonra şu üç satırı sırayla yazdırın: `a - Karmaşık(1, 1)`, `(a * a).büyüklüğü` ve `i² = -1` sınaması olarak `Karmaşık(0, 1) * Karmaşık(0, 1)`. Durum sınıfın kendi yazımı var: `Karmaşık(2.0,4.0)`; bunu iKojo da Koco da aynı basar.

ana.kojo — BAŞLANGIÇ

```
// KVI-3-I1 · Bir işleç daha
// Çıktı: a + a, a * a, a - Karmaşık(1, 1), (a * a).büyüklüğü, Karmaşık(0, 1) *
Karmaşık(0, 1)
durum sınıf Karmaşık(g: Kesir, s: Kesir) {
  tanım +(öbürü: Karmaşık) = Karmaşık(g + öbürü.g, s + öbürü.s)
  tanım *(öbürü: Karmaşık) =
    Karmaşık(g * öbürü.g - s * öbürü.s, g * öbürü.s + s * öbürü.g)
  tanım büyüklüğü = karekökü(g * g + s * s)
}
dez a = Karmaşık(1, 2)
satıryaz(a + a)
satıryaz(a * a)
```

çıktı — BEKLENEN-CİKTİ

```
Karmaşık(2.0,4.0)
Karmaşık(-3.0,4.0)
Karmaşık(0.0,1.0)
5.0
Karmaşık(-1.0,0.0)
```

Dizin: `alistirma-koco/kod/KVI-3-I1/`

ALIŞTIRMA KVI-3-A1

Kaçış adımları

Kitapçığın `kaçışAdımı`'nı beş noktada çalıştırın: (0, 0), (1, 1), (−1, 0), (0.3, 0.5), (−0.75, 0.1). Hangileri kümenin içinde (100)? Sonra adım sınırını 20 yapın: aynı noktalar için sonuç değişiyor mu? “İçinde” dediğimiz şeyin bir karar olduğunu görün.

çıktı — BEKLENEN-CİKTİ

```
(0.0, 0.0): 100
(1.0, 1.0): 2
(-1.0, 0.0): 100
(0.3, 0.5): 100
(-0.75, 0.1): 33
```

Dizin: `alistirma-koco/kod/KVI-3-A1/`

ZORLAYICI KVI-3-Z1

Mandelbrot, metinle

Boyamadan önce metinle çizin: 24 satır, 60 sütun; satır y'yi −1.2..1.2'ye, sütun x'i −2.2..0.8'e eşleyin. Her nokta için kaçış adımı: 100 ise `#`, 10'dan çoksa `+`, 4'ten çoksa `.`, yoksa boşluk. Satırları `satıryaz` ile basın. Beklenen çıktı bir resim; sizinki aynı mı? Sınırda farklı karakterler çıkarsa nedeni kayan noktalı sayılar olabilir, tartışın.

DÜŞÜNCE

Her satır bir y, her sütun bir x; kaçış adımı karaktere çevrilir.

YAKLAŞIM

`(0 |- 24).herbiriİçin { r => dez y = 1.2 - r * 2.4 / 23; satıryaz((0 |- 60).işle { c => karakter(kaçışAdımı(Karmaşık(-2.2 + c * 3.0 / 59, y))) }.yazıYap("")) }`.

İLK SATIR

```
tanım karakter(adım: Sayı): Harf = eğer (adım == 100) '#' yoksa eğer (adım > 10) '+' yoksa eğer (adım > 4) '.' yoksa ' '
```

çıktı — BEKLENEN-CİKTİ

```
                              ..+..
                             .....++..
                            ......+#+....
                           ......++####+......
                        ...+++..++++#####++.+.....
                     .........+##+##############+++#++..
                 ............+++##################++...
              ........+.....++#####################++.
            .......+#+++++++++#########################+..
         .......++##########+#########################+..
    ...........+#############################################+...
    ...........+#############################################+...
         .......++##########+#########################+..
            .......+#+++++++++#########################+..
              ........+.....++#####################++.
                 ............+++##################++...
                     .........+##+##############+++#++..
                        ...+++..++++#####++.+.....
                           ......++####+......
                            ......+#+....
                             .....++..
                              ..+..
```

Dizin: `alistirma-koco/kod/KVI-3-Z1/`

KEŞİF KVI-3-K1

Mandelbrot'u boyayın ~2 saat

Kitapçığın dördüncü son alıştırması: tuvalin her noktasını bir karmaşık sayıya çevirip kaçış adımına göre boyayın. Kaplumbağayla: `atla(x, y)`, rengi kur (`Renk.kym` ya da hazır renklerden adım sayısına göre seçin), `ileri(1)`. 200×200 nokta yeter; `canlandırmaHızınıKur(0)` ve `görünmez()` şart. Sonra pencerede yakınlaştırıp kenarına dalın; en güzel köşenin görüntüsünü sınıfla paylaşın. Bu dizin sizin.

DÜŞÜNCE

Her piksel bir karmaşık sayı; kaçış adımı rengi seçer.

YAKLAŞIM

İki döngü 200×200; `atla(x, y); kalemRenginiKur(renk(kaçışAdımı(c))); ileri(1)`. Renk için adım sayısını 0–255'e ölçekleyip `Renk(r, g, b)`.

İLK SATIR

```
(0 |- 200).herbiriİçin { px => (0 |- 200).herbiriİçin { py => ... } }
```

Tuvalde: Siyah bir Mandelbrot kümesi, kenarları mor-mavi-camgöbeği halkalarla; 200×200 nokta.

Dizin: `alistirma-koco/kod/KVI-3-K1/` — beklenen çıktı yok, çizim

VI.4

Yola devam

Kitapçık üç kolla bitiyor: Koco'da derinleşin, kardeş kitapçığa geçin, soru çözün; bir de bir şey inşa edin. Dört alıştırma: yaşam oyunu değişmez kümelerle, zıplayan kurbağa, ve iki açık uçlu Keşif.

1 1 2

YANIT YERİNE

kitapçık: Son söz yaşam oyunu temel-bilgiler 7: oyun tarifleri C++ cildi VI.6: yola devam

ALIŞTIRMA KVI-4-A1

Yaşam oyunu, kümeyle

Conway'in yaşam oyunu, değişmez kümelerle: canlı hücreler bir `Küme[(Sayı, Sayı)]`. Bir nesil: adayları (canlıların kendileri ve komşuları) al, her aday için canlı komşu sayısı; 3 ise canlı, 2 ise canlıysa canlı. `nesil(canlılar): Küme[...]` yazın; planörü dört nesil ilerletin ve hücreleri sıralı yazdırın: hepsi (1, 1) kaymış mı? Döngü yok, değişen yok; her nesil yeni bir küme.

çıktı — BEKLENEN-CİKTİ

```
başlangıç: (1,2) (2,3) (3,1) (3,2) (3,3)
4 nesil sonra: (2,3) (3,4) (4,2) (4,3) (4,4)
```

Dizin: `alistirma-koco/kod/KVI-4-A1/`

ZORLAYICI İKİLİ KVI-4-Z1

Zıplayan kurbağa

AtCoder DP A, kardeş kitapçığın klasiği: n taş, yükseklikler; kurbağa 1 ya da 2 taş atlar, bedel yükseklik farkının mutlak değeri; son taşa en ucuz bedel. Reçete: `bedel(i)` = i'nci taşa en ucuz varış; taban `bedel(0) = 0`; formül iki seçenekten küçüğü; kenar defteri. Yükseklikler 30 10 60 10 60 50 → 40. Gönderin.

DÜŞÜNCE

i'nci taşa varmak: i−1'den ya da i−2'den, farkın mutlak değeri bedel.

YAKLAŞIM

`bedel(0) = 0`, `bedel(1) = |h(1) - h(0)|`, gerisi `enUfağı(bedel(i-1) + |h(i) - h(i-1)|, bedel(i-2) + |h(i) - h(i-2)|)`; defterle.

İLK SATIR

```
tanım bedel(i: Sayı): Sayı = eğer (i == 0) 0 yoksa eğer (i == 1) (h(1) - h(0)).mutlak yoksa defter.alYoksa(i, { ... })
```

çıktı — BEKLENEN-CİKTİ

```
en ucuz: 40
```

Dizin: `alistirma-koco/kod/KVI-4-Z1/`

KEŞİF KVI-4-K1 Bir şey inşa edin ~bir hafta

Kitapçığın “bir şey inşa edin”i: yaşam oyununu tuvale alın (her hücre küçük bir kare; `canlandır` ile her karede bir nesil; `temel-bilgiler 7`'deki oyun tarifleri yol gösterir), ya da Koch tanesinin derinliğini klavyeden ayarlayan sürümü, ya da üç cisim benzetimini. Proje kodu sizinle kalır; arkadaşlarınıza izletin. Bu dizin sizin.

> **DÜŞÜNCE**
>
> Bir proje; tuvalde her karede bir adım.

> **YAKLAŞIM**
>
> Yaşam oyunu için: tahta `Dizin[Dizin[İkil]]`, bir `sonrakiNesil` işlevi (sekiz komşu say, dört kural), `canlandır` içinde çiz ve ilerlet.

> **İLK SATIR**
>
> `tanım sonrakiNesil(t: Dizin[Dizin[İkil]]): Dizin[Dizin[İkil]] = ...`

Dizin: `alistirma-koco/kod/KVI-4-K1/` — beklenen çıktı yok

KEŞİF KVI-4-K2 Haftada üç soru, iki dilde her hafta

Project Euler işlevsel üsluba bayılır; CSES algoritma kaslarını çalıştırır. Program: haftada üç soru, önce Koco'da, sonra aynı soruyu C++ cildindeki kardeşiyle. Başlangıç: PE 5, 7, 10; CSES 1068, 1083, 1069. Her birinde önce reçeteyi kâğıda yazın. Ve kitapçığın “en önemlisi”: öğrendiğiniz bir konuyu iki sayfada bir arkadaşınıza anlatın; çalışan kod, gerçek çıktı, bir de alıştırma.

> **DÜŞÜNCE**
>
> Haftada üç soru; her biri önce kâğıtta reçete, sonra iki dilde kod.

> **YAKLAŞIM**
>
> PE 5, 7, 10 işlevsel zincirlerle kısa; CSES 1068, 1083, 1069 için `satıroku` ile girdi okuyun ve C++ cildindeki kardeşiyle aynı girdide karşılaştırın.

> **İLK SATIR**
>
> `// soru: ...; durum: ...; taban: ...; formül: ...` yorumuyla başlayın.

Dizin: `alistirma-koco/kod/KVI-4-K2/` — beklenen çıktı yok

VI.5

Geri dönüş: çizelgeyi

eşlemle, engelli ızgara

Dinamik programlamanın çizelgesi bir dizi olmak zorunda değil; II. bölümün eşlemi de olur. İki kısa soru: ızgara yollarını defterle sayın, sonra ızgaraya engel koyun.

`2`

> **YANIT YERİNE**
>
> kitapçık II.3: Eşlem ve Belki kitapçık VI.2: Izgarada kaç yol var

ALIŞTIRMA KVI-5-A1 Çizelgeyi eşlemle tutun

`yol(s, k)`: $s \times k$ ızgarada sol üstten sağ alta (yalnız sağ ve aşağı) kaç yol var, `Uzun` olarak. Taban `s == 1 || k == 1`; defter bir `Eşlem[(Sayı, Sayı), Uzun]`, kitapçığın `alYoksa` kalıbıyla. 2×2, 5×5 ve 16×16 için yazdırın, son satırda defterde kaç kayıt biriktiğini (`sayı`). Aynı defteri üç soruda da kullanın; kayıt sayısı neden 256 değil de 225?

çıktı — BEKLENEN-CİKTİ

```
2x2: 2 yol
5x5: 70 yol
16x16: 155117520 yol
defterde 225 kayıt
```

Dizin: `alistirma-koco/kod/KVI-5-A1/`

ALIŞTIRMA KVI-5-A2 Engelli ızgarada yol sayma

Izgara bir `Dizin[Yazı]`; `#` engel. Sol üstten sağ alta yalnız sağa ve aşağı giderek kaç farklı yol var? Bu sefer çizelgeyi aşağıdan yukarı kurun: ilk satırda bir engelden sonrası 0; her karede `üst + sol`, engelse 0. Kitapçığın `soldanKatla` kalıbıyla satır satır, ya da bir `Eşlem[(Sayı, Sayı), Uzun]` ile yukarıdan aşağı; hangisini seçerseniz seçin, 4×5 harita için 4 çıkmalı.

çıktı — BEKLENEN-CİKTİ

```
sağ-aşağı yol sayısı: 4
```

Dizin: `alistirma-koco/kod/KVI-5-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`KVI-1-Z1` Zar dizileri

`KVI-4-Z1` Zıplayan kurbağa

`KVI-2-Z1` Aceleci ne zaman doğru?

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.